



Smart Moisture Advisor (SMA): An AI-Driven Irrigation Advisory System for Sugarcane

LY **Innovation-Based Major Project-B** Report

Submitted in partial fulfillment of the requirements of the Degree of
Bachelor of Technology in Computer Engineering

by

Kartik Verma

Kushal Soni

Supervisor

Dr. Shyamal Virnodkar



Department of Computer Engineering

K. J. Somaiya Institute of Technology

**An Autonomous Institute permanently affiliated to University of Mumbai
Ayurvihar, Sion, Mumbai -400022**

2026-27



Smart Moisture Advisor (SMA): An AI-Driven Irrigation Advisory System for Sugarcane

LY **Innovation-Based Major Project-B** Report

Submitted in partial fulfilment of the requirements of the Degree of
Bachelor of Technology in Computer Engineering

by

Kartik Verma (Roll No. _____)

Kushal Soni (Roll No. _____)

Supervisor

Dr. Shyamal Virnodkar



Department of Computer Engineering

K. J. Somaiya Institute of Technology

An Autonomous Institute permanently affiliated to University of Mumbai
Ayurvihar, Sion, Mumbai -400022
2026-27



CERTIFICATE



*This is to certify that the project entitled “**Smart Moisture Advisor (SMA): An AI-Driven Irrigation Advisory System for Sugarcane**” is bonafide work of **Kartik Verma, Kushal Soni** submitted to the University of Mumbai in partial fulfillment of the requirement in Major Project, for the award of the degree of “**Bachelors of Technology**” in “**Computer Engineering**”.*

Dr. Shyamal Virnodkar
Project Guide
Department of Computer Engineering

Dr. Sarita Ambadekar,
Head, Department of Computer Engineering

Dr. Vivek Sunnapwar
Principal KJSIT

Place: Sion, Mumbai-400022

Date:

PROJECT APPROVAL FOR L. Y.

This project report entitled “**Smart Moisture Advisor (SMA): An AI-Driven Irrigation Advisory System for Sugarcane**” by

Kartik Verma (Roll No. _____)

Kushal Soni (Roll No. _____)

is an approved Last Year **Innovation-Based Major Project in Computer Engineering**.

Examiners:

1. _____

Name and Signature

External Examiner

2. _____

Name and Signature

Internal Examiner

Place: Sion, Mumbai-400022

Date:

DECLARATION

We declare that this written submission represents our ideas in our own words and where other's ideas or words have been included, we have adequately cited and referenced the sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Kartik Verma

Kushal Soni

Date:

ACKNOWLEDGEMENT

Before presenting our LY Major Project work entitled “**Smart Moisture Advisor (SMA): An AI-Driven Irrigation Advisory System for Sugarcane**”, we would like to convey our sincere thanks to the people who guided us throughout the course for this project work.

First, We would like to express our immense gratitude towards our **Project Guide Dr. Shyamal Virnodkar** for the constant encouragement, support, guidance, and mentoring at the ongoing stages of the project and report.

We would like to express our sincere thanks to our **H.O.D. Dr. Sarita Ambadekar**, for the encouragement, co-operation, and suggestions progressing stages of the report.

We would like to express our sincere thanks to our beloved **Principal Dr. Vivek Sunnapwar** for providing various facilities to carry out this project.

Finally, we would like to thank all the teaching and non-teaching staff of the college, and our friends, for their moral support rendered during the course of the reported work, and for their direct and indirect involvement in the completion of our report work, which made our endeavor fruitful.

Place: Sion, Mumbai-400022

Date:

ABSTRACT

Sugarcane is among the most water-intensive crops in India, yet irrigation is usually scheduled from habit rather than from the crop's actual water need, causing water, energy and fertiliser waste or moisture stress and yield loss. For a sugar-factory network of 18,000–25,000 farmers, plot-level monitoring by field staff alone is impractical. This project presents the Smart Moisture Advisor (SMA), an AI-driven irrigation advisory system developed for the KJS-AGR-01 use case of K. J. Somaiya Institute of Technology with KIAAR and Godavari Biorefineries Ltd.

SMA uses 1,000 geofenced plots in Mandya district, Karnataka, with satellite-derived NDVI, LAI and soil moisture, soil pH, organic carbon and seasonal climate. As the data contain no irrigation or yield labels, a physics-informed approach is adopted: each plot is expanded into twelve seasonal scenarios labelled with the FAO-56 crop water balance and FAO-33 yield response plus field-realistic noise. Seven tree-ensemble models (Random Forest and Gradient Boosting) predict the next irrigation date with uncertainty, crop water requirement, irrigation depth, water-stress probability, cane yield, yield loss due to delayed irrigation and disease/pest risk, and a K-Means model groups plots into management zones. On plots never seen during training, the regression models achieve R^2 of 0.89–0.98 (next-irrigation error 0.69 days) and the classifiers 87–88% accuracy, outperforming linear and logistic baselines.

The trained models are exported to JSON and served natively by a Node.js/Express application whose predictions match scikit-learn exactly. A decision-support engine converts predictions into rainfall-adjusted irrigation advice, pump run-times, fertigation doses and explanations, and a Weighted-Shortest-Processing-Time scheduler allocates pumps within limited power windows. A web dashboard with farm maps, a what-if simulator and a multilingual (English, Kannada, Hindi, Marathi) chatbot delivers the advice. Under normal conditions the recommended schedule uses about 38% less water than fixed eight-day irrigation, and rain-aware advice postpones 894 irrigations when 40 mm of rain is forecast.

Keywords: Smart irrigation, sugarcane, machine learning, physics-informed learning, FAO-56, Random Forest, Gradient Boosting, decision support system, pump scheduling, multilingual advisory

CONTENTS

Chapter No.	TITLE	Page No.
	LIST OF FIGURES	viii
	LIST OF TABLES	x
	LIST OF ABBREVIATION	xi
1	INTRODUCTION	1
	1.1 Introduction	1
	1.2 Problem Definition	1
	1.3 Aim and Objectives	2
	1.4 Organization of the Report	3
2	REVIEW OF LITERATURE	4
	2.1 Literature Survey	4
	2.2 Research Gaps	8
3	REQUIREMENT SPECIFICATION	10
	3.1 Introduction	10
	3.2 Hardware requirements	10
	3.3 Software requirements	11
	3.4 Feasibility Study	11
	3.5 Cost Estimation	12
	3.6 Functional Requirements	13
	3.7 Non-Functional Requirements	14
	3.8 Project Schedule	14
4	PROJECT ANALYSIS & DESIGN	15
	4.1 Introduction	15
	4.2 UML Diagrams	15
	4.3 System Architecture	24
	4.4 Data Design	24
	4.5 User Interface Design	25
5	METHODOLOGY	26
	5.1 Introduction	26
	5.2 System Block Diagram	26
	5.3 Dataset Description and Exploratory Analysis	27
	5.4 Data Pre-processing	28

Chapter No.	TITLE	Page No.
	5.5 Physics-Informed Label Generation	29
	5.6 Machine-Learning Models	32
	5.7 Decision-Support Engine	34
	5.8 Pump Scheduling Algorithm	35
	5.9 Explainability Method	36
	5.10 Multilingual Advisory and Chatbot	36
6	IMPLEMENTATION DETAILS	37
	6.1 Introduction	37
	6.2 System Implementation	37
	6.3 Project Structure and Modules	42
	6.4 Training Pipeline Implementation	42
	6.5 Tree-Ensemble Inference in Node.js	43
	6.6 REST API	43
	6.7 Multilingual Advisory and Chatbot Implementation	44
	6.8 Security and Privacy Implementation	45
	6.9 Testing	45
	6.10 Deployment and Execution	46
7	RESULT ANALYSIS	47
	7.1 Introduction	47
	7.2 Performance Measures	47
	7.3 Comparative Analysis	55
8	CONCLUSION & FUTURE SCOPE	57
	8.1 Conclusion	57
	8.2 Future Scope	57
	REFERENCES	59
	PUBLISHED PAPERS	62
	CERTIFICATES	63
	PLAGIARISM REPORT	64

LIST OF FIGURES

Figure No.	Title	Page No.
1.1	Concept of the Smart Moisture Advisor: sense, predict, decide, advise and learn	1
2.1	Taxonomy of the reviewed literature	4
4.1	Use-case diagram of SMA	16
4.2	Level-0 data-flow diagram (context diagram)	17
4.3	Level-1 data-flow diagram	18
4.4	Class diagram	19
4.5	Sequence diagram: plot advisory request	20
4.6	Sequence diagram: pump schedule optimisation	20
4.7	State diagram of a plot's irrigation status	21
4.8	Activity diagram: model training pipeline	22
4.9	Activity diagram: advisory generation	22
4.10	Component diagram	23
4.11	Deployment diagram	23
5.1	System block diagram of SMA	27
5.2	Location of the 1,000 plots, coloured by taluk	27
5.3	FAO-56 sugarcane crop coefficient, expected NDVI and root depth versus crop age	30
5.4	Root-zone depletion with and without SMA-scheduled irrigation	31
6.1	Overview dashboard (summer dry spell scenario)	37
6.2	Farm map filtered to Maddur taluk, coloured by water stress	38
6.3	Farm Advisor: action banner, farmer advisory and probabilities for plot MM-MD-0110	38
6.4	Farm Advisor: moisture projection, yield-loss curve, fertigation, explanation and feedback	39
6.5	What-If Simulator with the "Heat wave, dry soil" preset	40
6.6	Pump Scheduler for village Hosur with two power windows and 20 pumps per feeder	40
6.7	Model Insights: pipeline, model accuracy cards, feature importance and predicted-versus-actual plot	41
6.8	Krishi Mitra chatbot answering in English and Kannada	41
7.1	R ² of the SMA models compared with linear-regression baselines	48
7.2	Predicted versus actual values for four regression models (held-out plots)	48
7.3	Confusion matrices of the water-stress and disease-risk classifiers	49

Figure No.	Title	Page No.
7.4	Top feature importances of three models	50
7.5	Advisory outcomes and weekly irrigation water under four weather scenarios	51
7.6	Weekly water requirement by irrigation method	52
7.7	Plot MM-MD-0110: (a) root-zone projection, (b) yield loss versus irrigation delay	52
7.8	Pump scheduling: (a) estimated cane loss and (b) high-stress plots served, for different feeder capacities	53
7.9	Learning curve of the next-irrigation model (held-out plots)	54

LIST OF TABLES

Table No.	Title	Page No.
2.1	Summary of the literature survey	7
2.2	Research gaps and how SMA addresses them	9
3.1	Hardware requirements for development and model training	10
3.2	Hardware requirements for deployment	11
3.3	Software requirements	11
3.4	Size of the developed software	12
3.5	Cost estimation	13
3.6	Functional requirements	13
3.7	Non-functional requirements	14
4.1	Description of the main use cases	16
4.2	Data dictionary of the IrrigationAdvisoryDataset	24
5.1	Descriptive statistics of the plot features	28
5.2	Scenario augmentation parameters	29
5.3	Sugarcane crop-coefficient curve and stage sensitivity	30
5.4	Model configuration and hyperparameters	33
5.5	Stage-wise fertigation split of the NPK package	35
6.1	Project modules	42
6.2	REST API endpoints	43
6.3	The same advisory generated in four languages (plot MM-MD-0110, rain forecast)	44
6.4	Automated test cases	45
7.1	Performance of the regression models on unseen plots	47
7.2	Performance of the classification models on unseen plots	49
7.3	System-level results for the 1,000 plots under four weather scenarios	50
7.4	Pump-scheduling results for village Hosur	53
7.5	Computational performance	53
7.6	Performance on held-out plots by taluk	54
7.7	Comparison of SMA with existing approaches from the literature	55
7.8	Quantitative comparison with baselines	56

LIST OF ABBREVIATIONS

Sr. No	Abbreviation	Description
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	CSV	Comma-Separated Values
4	DFD	Data Flow Diagram
5	DPDP	Digital Personal Data Protection (Act, 2023)
6	DSS	Decision Support System
7	ET	Evapotranspiration
8	ET ₀	Reference Evapotranspiration
9	ET _c	Crop Evapotranspiration (crop water requirement)
10	FAO	Food and Agriculture Organization of the United Nations
11	FC	Field Capacity
12	FCFS	First-Come, First-Served
13	GBL	Godavari Biorefineries Ltd.
14	GBM	Gradient Boosting Machine
15	GIS	Geographic Information System
16	HTTP	HyperText Transfer Protocol
17	IMD	India Meteorological Department
18	IoT	Internet of Things
19	JSON	JavaScript Object Notation
20	K _c	Crop Coefficient
21	KIAAR	K. J. Somaiya Institute of Applied Agricultural Research
22	K _s	Water-Stress Coefficient
23	K _y	Yield Response Factor
24	LAI	Leaf Area Index
25	LLM	Large Language Model
26	LOC	Lines of Code
27	MAD	Median Absolute Deviation
28	MAE	Mean Absolute Error
29	ML	Machine Learning
30	NDVI	Normalized Difference Vegetation Index
31	NPK	Nitrogen, Phosphorus, Potassium
32	OC	Organic Carbon
33	PWP	Permanent Wilting Point
34	R ²	Coefficient of Determination
35	RAW	Readily Available Water
36	REST	Representational State Transfer

Sr. No	Abbreviation	Description
37	RF	Random Forest
38	SMA	Smart Moisture Advisor
39	SMS	Short Message Service
40	SPA	Single-Page Application
41	STEPS	Somaiya Technology for Efficient Plot Survey
42	TAW	Total Available Water
43	UI	User Interface
44	UML	Unified Modeling Language
45	WSPT	Weighted Shortest Processing Time

CHAPTER 1

INTRODUCTION

1.1 Introduction

Sugarcane is one of India's most water-intensive crops, needing roughly 1,500–2,500 mm of water per season [1], mostly supplied from canals and borewells. KIAAR and Godavari Biorefineries Ltd. (GBL) support about 18,000–25,000 sugarcane farmers across Northern Karnataka [2], whose plots have already been geofenced and whose soil and crop records are held in platforms such as STEPS (Somaiya Technology for Efficient Plot Survey).

Satellite remote sensing, the Internet of Things (IoT), weather forecasting and Machine Learning (ML) now allow crop vigour, soil moisture and climate to be estimated for every plot remotely [3], [4], [5]; agronomy provides the equations that turn them into crop water requirements [6], and ML is widely used for yield, disease and water management [7], [8]. The domain of this project is therefore AI-driven precision irrigation and decision support for sugarcane.

This report presents the Smart Moisture Advisor (SMA), an AI irrigation advisory system built for the KJS-AGR-01 use case, combining physics-informed machine learning, a decision engine, pump scheduling and a multilingual web platform. Fig. 1.1 summarises the idea: SMA senses plot conditions, predicts water needs and risks with seven ML models, decides on concrete actions, advises the farmer in their own language and learns from the farmer's feedback.

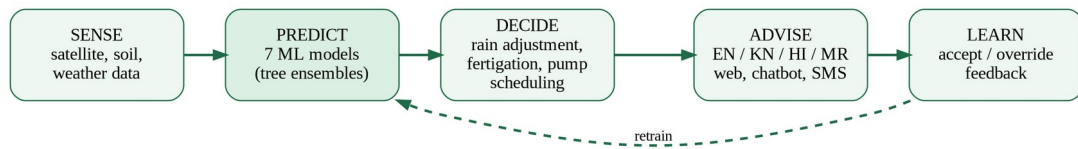


Fig. 1.1: Concept of the Smart Moisture Advisor: sense, predict, decide, advise and learn

1.2 Problem Definition

Sugarcane irrigation scheduling in the target region is largely based on farmers' experience rather than on the actual water requirement of the crop. Because of limited knowledge of crop water requirements, soil-moisture conditions, weather forecasts and

irrigation timing, farmers frequently over-irrigate or delay irrigation [2]. The consequences reported by the use case are:

- **Water wastage** through over-irrigation, which depletes canals and groundwater.
- **Reduced productivity and yield loss** due to moisture stress when irrigation is delayed, particularly during tillering and grand growth, when sugarcane is most sensitive to water deficit [9].
- **Increased energy consumption** and inefficient use of the limited electricity supplied to irrigation pumps.
- **Higher cultivation costs and nutrient losses** caused by improper fertigation and leaching.
- **Difficulty in monitoring** thousands of spatially distributed plots with a limited number of field staff.

The problem addressed by this project can be stated formally as follows. For every geofenced plot i with observed satellite, soil and climate features x_i , an estimated crop age a_i and a weather scenario w (forecast rain, temperature and humidity anomalies, dry days since the last observation), the system must estimate: (a) the number of days until irrigation is required and its uncertainty, (b) the crop water requirement and irrigation depth, converted into water volume and pump run-time for the plot's irrigation method, (c) the probability of water stress and of disease or pest risk, (d) the expected yield and the yield loss that would result from delaying irrigation, and (e) a fertigation dose for the current growth stage. These are then converted into a power-constrained pump schedule and plain-language advice in English, Kannada, Hindi or Marathi.

A key constraint is that the provided dataset of 1,000 geofenced plots has no ground-truth labels (no irrigation logs, volumes or yields), so the system must combine agronomic knowledge with machine learning.

1.3 Aim and Objectives

Aim: To design and implement an AI-enabled irrigation advisory system for sugarcane that integrates satellite-derived plot observations, soil properties, weather scenarios and crop-specific requirements to generate plot-specific, explainable and multilingual irrigation, fertigation and pump-scheduling recommendations.

Objectives:

1. To audit, clean and analyse the dataset of 1,000 sugarcane plots and identify its limitations.
2. To develop a physics-informed labelling engine based on the FAO-56 crop water balance and the FAO-33 yield-response relationship, so that supervised models can be trained without field labels.
3. To train and validate ML models for next-irrigation date, crop water requirement, irrigation depth, water-stress probability, yield, yield loss due to delayed irrigation and disease/pest risk, and to compare them against baseline models on plots not seen during training.
4. To build a decision-support engine that provides rainfall-adjusted irrigation advice, pump run-time, fertigation doses and explanations of each recommendation.
5. To optimise pump scheduling under limited electricity availability using a weighted scheduling algorithm and compare it with a conventional first-come rotation.
6. To develop a farmer-friendly web platform with a region dashboard, geofenced farm map, plot advisor, what-if simulator, pump scheduler and a multilingual chatbot, supporting human-in-the-loop feedback.
7. To follow responsible-AI principles and enable retraining on real KIAAR/GBL field data.

1.4 Organization of the Report

The report is organised as follows. Chapter 2 reviews the literature and identifies research gaps. Chapter 3 specifies requirements, feasibility, cost and schedule. Chapter 4 presents the analysis and design with UML and data-flow diagrams, architecture and data design. Chapter 5 describes the methodology — block diagram, data analysis, physics-informed labelling, models, decision engine, pump scheduling and explainability — with its mathematical formulation. Chapter 6 explains the implementation with screenshots, modules, API and testing. Chapter 7 analyses the results and compares SMA with existing systems, and Chapter 8 concludes with the future scope.

CHAPTER 2

REVIEW OF LITERATURE

2.1 Literature Survey

Survey strategy. The literature survey was carried out in three steps. First, the KJS-AGR-01 use-case document [2] was analysed to extract the required AI models, data sources and governance requirements. Second, peer-reviewed literature was searched on IEEE Xplore, ScienceDirect, MDPI and Google Scholar using combinations of the keywords smart irrigation, irrigation scheduling, sugarcane water requirement, crop evapotranspiration, soil moisture remote sensing, machine learning in agriculture, yield prediction, decision support system and explainable AI. Foundational standards were taken directly from the Food and Agriculture Organization (FAO) irrigation and drainage papers. Third, works were selected if they (i) proposed a method or system for irrigation or crop decision support, (ii) provided an algorithm or dataset used in this project, or (iii) defined an evaluation, scheduling or governance principle that the project follows. The selected works were grouped into the five themes shown in Fig. 2.1 and are summarised in text below and in tabular form in Table 2.1.

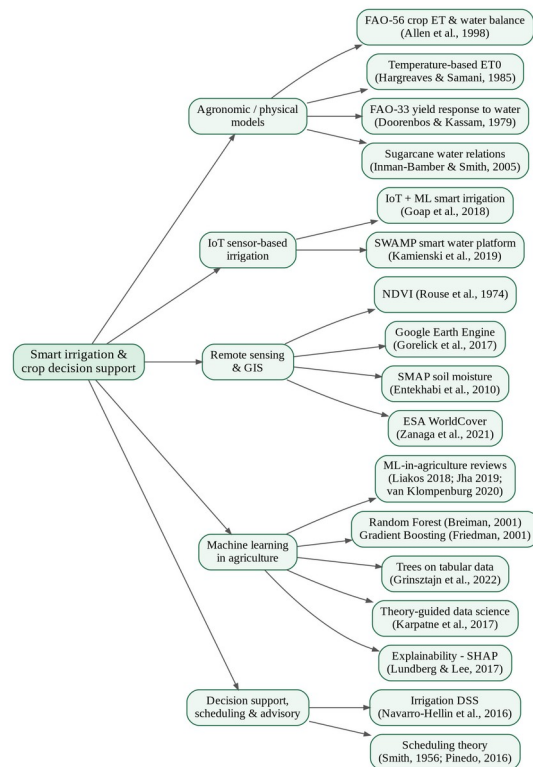


Fig. 2.1: Taxonomy of the reviewed literature

2.1.1 Agronomic and physical models of crop water use

The FAO Irrigation and Drainage Paper 56 by Allen et al. [6] is the international reference for computing crop water requirements. It defines the reference evapotranspiration (ET_0), crop coefficients (K_c) that vary with growth stage, and a root-zone soil-water balance in which total available water (TAW), readily available water (RAW) and a water-stress coefficient (K_s) determine when irrigation is needed. Because full Penman–Monteith inputs are rarely available at plot level, Hargreaves and Samani [10] proposed a temperature-based ET_0 equation that requires only temperature and extraterrestrial radiation; it is widely used where weather stations are sparse. The consequences of water deficit on production are captured by the FAO-33 yield-response relationship of Doorenbos and Kassam [11], which links the relative yield decrease to the relative evapotranspiration deficit through a yield-response factor K_y (about 1.2 for sugarcane). For sugarcane specifically, Inman-Bamber and Smith [9] reviewed how water deficits reduce canopy expansion and stalk elongation, showing that sensitivity is highest during the active growth phases. FAO also summarises the seasonal water needs and growth stages of the crop [1]. These models provide a sound scientific basis for scheduling but are deterministic, require complete inputs, and do not learn from data or express uncertainty.

2.1.2 IoT sensor-based smart irrigation

Goap et al. [12] built an IoT-based smart irrigation system that combines soil-moisture, temperature and humidity sensors with weather-forecast data, and uses machine learning to predict soil moisture and automate irrigation. The work showed that forecasts can prevent unnecessary irrigation, but the approach requires sensors in every field and was demonstrated at the scale of a single field. Kamienski et al. [13] described SWAMP, an IoT-based smart water management platform for precision irrigation evaluated in pilots with different crops and countries; the emphasis was on a configurable platform architecture rather than on crop-specific predictive models or farmer-facing advisories. Both works confirm the value of connected data, while highlighting the cost of dense sensor networks when thousands of smallholder plots must be covered — precisely the situation of the GBL/KIAAR network.

2.1.3 Remote sensing and geospatial data

Remote sensing offers plot-level observations without in-field hardware. The Normalized Difference Vegetation Index (NDVI), introduced by Rouse et al. [3],

remains the most widely used indicator of green biomass and crop vigour. Satellite missions such as SMAP [4] provide regular soil-moisture observations, although at coarse resolution and only for the surface layer, so the root-zone water state must be modelled. Cloud platforms such as Google Earth Engine [5] make it possible to extract such indices for thousands of plots, and global land-cover products such as ESA WorldCover [14] are used to locate cropland; the dataset of this project was sampled on the WorldCover cropland class. Remote sensing therefore supplies the inputs for large-scale advisory, but by itself does not produce irrigation decisions.

2.1.4 Machine learning in agriculture

Liakos et al. [7] reviewed machine learning in agriculture across crop, livestock, water and soil management, and Jha et al. [8] surveyed AI-based automation, including irrigation. Both reviews report wide use of ML for individual prediction tasks, but few systems integrate several predictions into a single operational advisory. In their systematic review of ML-based crop yield prediction, van Klompenburg et al. [15] found that temperature, rainfall and soil type are the most frequently used features and that many algorithms perform competitively, yet yield prediction is usually studied in isolation from irrigation decisions. Regarding model choice, Random Forests [16] and Gradient Boosting Machines [17] are ensemble methods of decision trees that are accurate and robust on tabular data; Grinsztajn et al. [18] showed that tree-based models still outperform deep neural networks on typical medium-sized tabular datasets. Reliable implementations are available in scikit-learn [19], and standard evaluation methodology is described by Hastie et al. [20]. When labelled data are scarce, Karpatne et al. [21] argue for theory-guided data science, in which scientific knowledge constrains or generates the training data — the principle adopted by SMA. For trust and accountability, Lundberg and Lee [22] proposed SHAP explanations; SMA uses a lighter, model-agnostic sensitivity explanation that can run for every request.

2.1.5 Decision support, scheduling and governance

Navarro-Hellín et al. [23] developed a smart irrigation decision support system that uses soil and climate sensor data with machine-learning regression to estimate weekly irrigation needs, reproducing expert decisions closely; however, the approach depends on an expert-labelled irrigation history. Allocation of pumping time under limited electricity is a scheduling problem: Smith's rule [24] (Weighted Shortest Processing Time, WSPT) is optimal for minimising total weighted completion time on a single

machine, and list-scheduling heuristics for parallel machines are well established [25]. For data quality, the modified z-score of Iglewicz and Hoaglin [26] gives a robust rule for flagging outliers, and K-means clustering [27] is a standard way of grouping observations into zones. Finally, the Digital Personal Data Protection Act, 2023 [28] sets the legal requirements of consent, purpose limitation and data minimisation that any farmer-data platform in India must follow, and the COCOMO model of Boehm [29] is the classical basis for estimating software development effort.

Table 2.1 summarises every reference used in this report, including the approach, the key contribution and the limitation relevant to this project.

Table 2.1: Summary of the literature survey

Ref.	Authors / Year	Approach	Key contribution / finding	Limitation w.r.t. this project
[1]	FAO (crop information)	Agronomic summary of sugarcane water needs and stages.	Crop needs $\approx 1,500\text{--}2,500$ mm per season.	Generic; not plot-specific.
[2]	Virnodkar et al. (KJSIT, KIAAR, GBL), 2026	Use case KJS-AGR-01 for an AI irrigation advisory; lists 11 required models.	Defines problem, data, workflow and governance.	Specification only; no implementation.
[3]	Rouse et al., 1974	Introduced NDVI from red and NIR reflectance.	Satellite indicator of crop vigour.	Index only; no decisions.
[4]	Entekhabi et al., 2010	SMAP satellite soil-moisture mission.	Regular soil-moisture maps.	Coarse, surface layer only.
[5]	Gorelick et al., 2017	Google Earth Engine cloud platform.	Plot-level indices at scale.	Data platform only.
[6]	Allen et al., 1998 (FAO-56)	FAO-56 ET, crop coefficients and root-zone water balance.	Standard basis of irrigation scheduling.	Needs full inputs; no learning or uncertainty.
[7]	Liakos et al., 2018	Review of ML in agriculture.	ML widely used for yield, disease, water.	Few integrated advisories.
[8]	Jha et al., 2019	Review of AI-based farm automation.	Automation improves efficiency.	Calls for low-cost, farmer-friendly tools.
[9]	Inman-Bamber & Smith, 2005	Review of sugarcane water relations.	Growth most sensitive to deficit in active growth.	Physiology only; no tool.
[10]	Hargreaves & Samani, 1985	Temperature-based ET ₀ equation.	ET ₀ from temperature and radiation only.	Less accurate in humid climates.
[11]	Doorenbos & Kassam, 1979 (FAO-33)	FAO-33 yield response factor K_y .	Links ET deficit to yield loss (sugarcane $K_y \approx 1.2$).	Seasonal; does not schedule irrigation.
[12]	Goap et al., 2018	IoT sensors + weather forecast + ML to automate irrigation.	Forecast-aware automatic irrigation.	Sensors per field; single field; no yield-loss or scheduling.
[13]	Kamienski et al., 2019 (SWAMP)	SWAMP IoT platform for precision irrigation.	Configurable smart-water platform in pilots.	Limited ML; no local-language advisory.
[14]	Zanaga et al., 2021	ESA WorldCover 10 m land cover.	Cropland mask used for sampling.	No crop or management data.

Ref.	Authors / Year	Approach	Key contribution / finding	Limitation w.r.t. this project
[15]	van Klompenburg et al., 2020	Review of ML yield prediction.	Temperature, rainfall, soil most used.	Yield not linked to irrigation.
[16]	Breiman, 2001	Random Forest ensemble of trees.	Robust, with feature importance.	Generic algorithm.
[17]	Friedman, 2001	Gradient Boosting of trees.	Accurate on tabular data.	Needs tuning and validation.
[18]	Grinsztajn et al., 2022	Benchmark of trees vs deep learning.	Trees best on medium tabular data.	Justifies model choice.
[19]	Pedregosa et al., 2011	scikit-learn ML library.	Reliable RF, GBM, K-Means.	Python runtime; SMA exports JSON.
[20]	Hastie et al., 2009	Statistical learning textbook.	Evaluation and ensemble theory.	Methodology reference.
[21]	Karpatne et al., 2017	Theory-guided data science.	Theory helps when labels are scarce.	Conceptual; motivates SMA labels.
[22]	Lundberg & Lee, 2017	SHAP feature attributions.	Consistent local explanations.	Costly; SMA uses lighter method.
[23]	Navarro-Hellín et al., 2016	Sensor data + ML regression for weekly irrigation needs.	Reproduced expert decisions closely.	Needs labelled history and sensors.
[24]	Smith, 1956	WSPT single-machine scheduling rule.	Optimal weighted completion time.	No power windows or feeders.
[25]	Pinedo, 2016	Scheduling theory textbook.	Fast list-scheduling heuristics.	Generic; adapted to pumps.
[26]	Iglewicz & Hoaglin, 1993	Modified z-score outlier test.	Robust flag at $ z > 3.5$.	Used for sensor-quality flags.
[27]	MacQueen, 1967	K-means clustering.	Simple, scalable partitioning.	Used for management zones.
[28]	Government of India, 2023	Indian data-protection law.	Consent and data minimisation.	Guides privacy design.
[29]	Boehm, 1981	COCOMO effort model.	Effort from code size.	Used for cost estimation.
[30]	OpenJS Foundation	Node.js runtime and Express web framework.	Lightweight, event-driven API server.	Used to serve SMA models and API.
[31]	Agafonkin et al.; Chart.js contributors	Open-source web mapping and charting libraries.	Interactive maps and charts in the browser.	Used for the SMA farm map and dashboards.

2.2 Research Gaps

From the survey, the following research gaps were identified for a large-scale sugarcane irrigation advisory such as the one required by KIAAR and GBL:

1. **No integrated, multi-model advisory.** Existing works predict soil moisture, irrigation need or yield separately; very few combine irrigation timing, duration, water stress, yield loss from delay, fertigation and disease risk into one consistent recommendation [7], [15].
2. **Dependence on dense sensors or labelled history.** IoT systems [12], [13] need hardware in every field, and ML decision-support systems [23] need expert-

labelled irrigation logs — neither is available for thousands of smallholder plots today.

3. **Deterministic agronomic models without uncertainty.** FAO-based schedulers [6] give a single answer and cannot express confidence or learn from field feedback.
4. **Weather-blind schedules.** Fixed-interval irrigation ignores forecast rain, and few systems quantify the water saved by postponing irrigation.
5. **Energy constraints ignored.** Rural pump sets receive three-phase power only in limited windows, but irrigation advice is rarely coupled with pump scheduling [24], [25].
6. **Limited explainability and farmer accessibility.** Many systems present technical outputs in English only, without explanations, voice output or local-language support, which limits farmer trust and adoption [8].
7. **Governance.** Privacy, human oversight and continuous learning from farmer feedback are rarely designed into prototype systems [28].

Table 2.2 maps each gap to the way it is addressed in SMA.

Table 2.2: Research gaps and how SMA addresses them

Research gap	How SMA addresses it
Isolated single-task models	Seven coordinated models (next irrigation, water requirement, depth, stress, yield, yield loss, disease risk) plus K-Means zones feed one decision engine and one advisory.
Need for sensors / labelled history	Physics-informed labels from FAO-56 and FAO-33 on 12,000 seasonal scenarios; the same pipeline retrains on real labels when they become available.
No uncertainty in agronomic models	Random-Forest tree spread gives $\pm$ days; class probabilities are shown instead of single labels.
Weather-blind schedules	Forecast rain is a model input; a "no-rain" counterfactual converts advice into "skip" and quantifies rain used.
Energy constraints ignored	WSPT pump scheduler within power windows and feeder capacity, compared with a first-come rotation.
Low explainability / accessibility	Per-feature explanations, 14-day moisture projection, advisories in English, Kannada, Hindi and Marathi, read-aloud and chatbot.
Weak governance	Anonymised plot IDs, input validation, accept/override feedback log for retraining, documented limitations.

CHAPTER 3

REQUIREMENT SPECIFICATION

3.1 Introduction

This chapter specifies what the Smart Moisture Advisor must do and what it needs in order to do it. The requirements were derived from the KJS-AGR-01 use case [2], from discussions of the expected users (farmers, field supervisors, agronomists, sugar-factory cane departments and data scientists) and from the constraints of the provided dataset. The hardware and software requirements are listed separately for development, for the application server and for end-user devices. The chapter then states the functional and non-functional requirements, assesses technical, economic, operational, schedule and legal feasibility, estimates the cost of the project with the COCOMO model [29] and presents the project schedule.

A guiding decision was to keep the deployed system light and offline-capable: models are trained once in Python and exported as JSON, so the application needs only Node.js and a browser and can run on an ordinary office laptop.

3.2 Hardware requirements

The hardware requirements for development and model training are given in Table 3.1. Training takes about 75 seconds on a four-core CPU; no GPU is required.

Table 3.1: Hardware requirements for development and model training

Component	Minimum	Used / recommended
Processor	Dual-core x86-64, 2.0 GHz	Quad-core x86-64 (4 vCPU)
Memory (RAM)	4 GB	8–16 GB
Storage	2 GB free (code, dependencies, models)	10 GB SSD
Network	Needed only to install packages	Broadband for map tiles and updates

The application server hosts the Node.js API and the static web application. Because inference for all 1,000 plots completes in about 0.2 seconds, a small machine is sufficient (Table 3.2). End users only need a device with a modern web browser.

Table 3.2: Hardware requirements for deployment

Tier	Specification	Purpose
Application server	2 vCPU, 2–4 GB RAM, 5 GB disk (laptop, office PC or cloud VM)	Runs Node.js API, model inference, scheduler and serves the web app
Supervisor / office client	PC or laptop with a modern browser, 4 GB RAM	Dashboard, farm map, pump scheduler, model insights
Farmer / field-staff client	Android smartphone (2 GB RAM) with Chrome, or feature phone for SMS	Plot advisory, chatbot, read-aloud; SMS text for basic phones

3.3 Software requirements

All software is free and open-source and runs on Windows, Linux and macOS; the requirements are listed in Table 3.3. PlantUML, Graphviz and Matplotlib were used for diagrams, and Git/GitHub for version control.

Table 3.3: Software requirements

Category	Software / library	Version	Role in SMA
Runtime	Node.js	18 or newer (22 used)	Application server and ML inference [30]
Web framework	Express	5.x	REST API and static file server
Data parsing	csv-parse	7.x	Reading the plot dataset
Visualisation	Chart.js, Leaflet	4.x, 1.9	Charts and interactive farm map [31]
ML training	Python, NumPy, pandas, scikit-learn	3.9+, 2.x, 2.x, 1.3+	Label generation, training, evaluation, export [19]
Testing	node:test (built-in)	Node 18+	Unit, parity and API tests

3.4 Feasibility Study

A feasibility study was carried out to confirm that the system can be built and used within the constraints of the project. The five dimensions considered are summarised below.

Technical feasibility. The required data are in the dataset and can be refreshed from satellite platforms [5]; the missing labels are replaced by physics-informed labels from FAO-56 and FAO-33 [6], [11], [21]; models train in about a minute and serve in milliseconds; and the stack is mature. The project is technically feasible.

Economic feasibility. All software is open-source and the system runs on existing laptops or a small cloud virtual machine. The main costs are development effort and, for a field deployment, optional sensors and SMS charges. The potential benefits — about 38% less irrigation water under normal conditions (Chapter 7), lower pump electricity use and reduced yield loss — are recurring, so the system is economically attractive.

Operational feasibility. Farmers receive short instructions such as "Start irrigating this morning: about 22 hours of pumping" in their own language, with voice output for low-literacy users. Supervisors use a familiar dashboard and map. Human-in-the-loop accept/override buttons keep experts in control, which supports adoption.

Schedule feasibility. The work was divided into ten phases over one semester (Section 3.8). Using a light stack and pre-trained models allowed the complete system to be built, tested and documented within the available time.

Legal and ethical feasibility. The dataset contains no personal data; plots are identified by anonymised IDs. Feedback logs store only plot ID, role and decision, in line with the data-minimisation principle of the DPDP Act, 2023 [28].

3.5 Cost Estimation

The development effort was estimated with the basic COCOMO model [29] for an organic project (small team, familiar environment, flexible requirements). The size of the delivered software, counted as non-blank lines of code, is shown in Table 3.4.

Table 3.4: Size of the developed software

Module	Language	Lines of code
Server: API, data loader, inference, decision engine, scheduler, chatbot	JavaScript	1,332
Web application: pages, charts, map, styles	JavaScript, HTML, CSS	1,765
ML pipeline: agronomy engine, training, export	Python	474
Automated tests	JavaScript	158
Total		3,729 (≈ 3.73 KLOC)

For an organic project the effort E (person-months), development time D (months) and average staffing S are given by:

$$E = 2.4 \times (\text{KLOC})^{1.05} = 2.4 \times (3.73)^{1.05} \approx 9.56 \text{ person-months} \quad (3.1)$$

$$D = 2.5 \times E^{0.38} = 2.5 \times (9.56)^{0.38} \approx 5.9 \text{ months} \quad (3.2)$$

The average staffing $E/D \approx 1.6$ persons over about six months matches a two-member team working part-time. The notional development cost and the indicative running cost of a deployment are given in Table 3.5. Market rates are indicative and should be confirmed at the time of deployment.

Table 3.5: Cost estimation

Item	Basis	Estimated cost (₹)
Development effort (notional)	9.56 person-months $\times$ ₹25,000 per person-month	2,39,000
Software licences	Open-source stack	0
Development hardware	Existing student laptops	0
Dataset	Provided with the use case	0
Application hosting (optional)	Small cloud VM, 2 vCPU / 4 GB, $\approx$ ₹1,500–3,000 per month	18,000–36,000 per year
SMS advisories (optional)	1,000 plots $\times$ 2 SMS/week $\times$ 52 weeks at $\approx$ ₹0.20	$\approx$ 20,800 per year
Field sensors (future pilot)	Soil-moisture nodes and gateway for a pilot village	To be quoted

3.6 Functional Requirements

The functional requirements define the services SMA must provide. They are listed in Table 3.6 with their priority.

Table 3.6: Functional requirements

ID	Requirement	Priority
FR1	Load plot records (features, polygons) from CSV, impute missing NDVI and flag outliers.	High
FR2	Predict days to next irrigation with uncertainty for every plot.	High
FR3	Predict crop water requirement (ETc) and irrigation depth; convert to volume and pump hours for the chosen irrigation method and pump flow.	High
FR4	Predict water-stress probabilities (None, Mild, Moderate, Severe) and disease/pest risk (Low, Medium, High).	High
FR5	Predict yield and yield loss for irrigation delays of 0–14 days.	High
FR6	Adjust advice for forecast rain and report irrigations that can be skipped.	High
FR7	Generate a stage-wise fertigation plan (urea, 12-61-0, MOP) with soil-based notes.	Medium
FR8	Optimise pump schedules within power windows and feeder capacity; compare with rotation.	High
FR9	Generate advisories in English, Kannada, Hindi and Marathi, with read-aloud and SMS copy.	High
FR10	Provide a chatbot that answers plot questions in the four languages.	Medium

ID	Requirement	Priority
FR11	Show a region dashboard, a farm map with prediction layers and a what-if simulator.	High
FR12	Explain each recommendation and show model accuracy, importance and confusion matrices.	Medium
FR13	Record accept/override feedback for retraining.	Medium
FR14	Retrain all models from data with one command and verify inference parity.	Medium

3.7 Non-Functional Requirements

Table 3.7: Non-functional requirements

Category	Requirement
Performance	Score all 1,000 plots in under 1 s; single-plot advisory in under 100 ms on a laptop.
Accuracy	Each model must outperform its linear/logistic baseline on unseen plots.
Scalability	Stateless API with per-scenario caching; linear scaling to 25,000 plots.
Usability	Simple language, local languages, voice output, responsive layout for phones, light/dark themes.
Accessibility	Colour-blind-safe chart palette; status shown with icons and labels, not colour alone.
Security	Validate and clamp all inputs; no secrets in code; restrict request size.
Privacy	Anonymised plot IDs; no personal data; minimal feedback log.
Maintainability	Modular code, documented API, automated tests, reproducible training; runs offline on Windows, Linux or macOS.

3.8 Project Schedule

The project was planned in ten overlapping phases over about sixteen weeks: problem study and use-case analysis (weeks 1–2), literature survey (2–4), dataset audit and cleaning (3–4), agronomy engine (4–6), model training and evaluation (6–8), Node.js inference and API (7–9), decision engine and scheduler (9–11), web dashboard and chatbot (10–13), testing and validation (13–14), and report, presentation and review (14–16).

CHAPTER 4

PROJECT ANALYSIS & DESIGN

4.1 Introduction

This chapter presents the analysis and design of the Smart Moisture Advisor. The system is modelled with the Unified Modeling Language (UML) and with data-flow diagrams: the use-case diagram identifies the actors and the services they need; data-flow diagrams show how data moves from satellite observations to farmer advisories; the class diagram describes the main software abstractions; sequence diagrams describe the three most important interactions; the state diagram models the irrigation life-cycle of a plot; activity diagrams describe the training pipeline and the advisory logic; and component and deployment diagrams describe the software structure and its physical deployment. The chapter concludes with the layered system architecture, the data design and the user-interface design.

4.2 UML Diagrams

4.2.1 Use-case diagram

The use-case diagram in Fig. 4.1 shows five actors. The Farmer views the plot advisory, chooses a language and listens to it, asks the Krishi Mitra chatbot and accepts or overrides advice. The Field Supervisor monitors the region dashboard and farm map, optimises pump schedules and records overrides. The Agronomist sets weather scenarios, runs what-if simulations and reviews model explanations. The Factory / Cane Department uses the dashboard and pump schedules for planning, and the Data Scientist / Administrator updates the dataset and retrain the models. The central use case Generate predictions is included by every use case that needs model output, which reflects the design decision that all views are driven by the same seven models and are therefore mutually consistent.

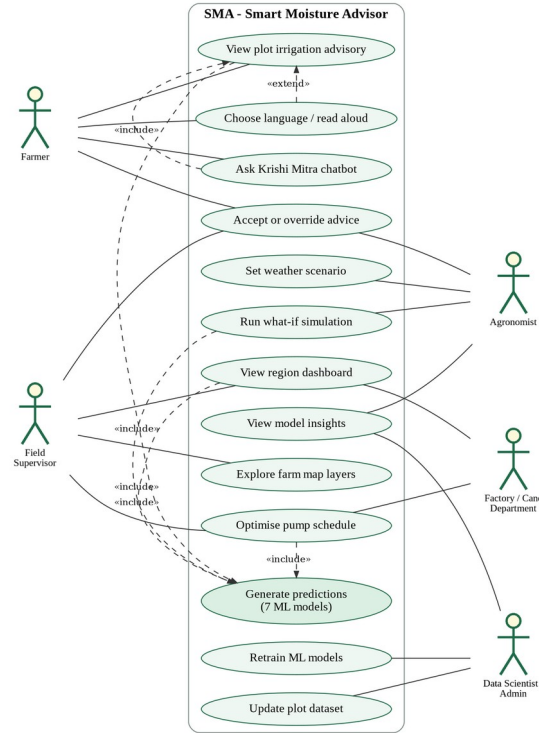


Fig. 4.1: Use-case diagram of SMA

The main use cases are described in Table 4.1.

Table 4.1: Description of the main use cases

Use case	Actor(s)	Description	Outcome
View plot advisory	Farmer	Enter a plot ID; the system predicts irrigation date, duration, stress, yield, loss from delay, fertigation and risk under the current scenario.	Advisory card, charts and plain-language text
Choose language / read aloud	Farmer	Switch between English, Kannada, Hindi and Marathi; speech synthesis reads the advice.	Localised advice
Ask Krishi Mitra	Farmer	Type a question in any supported language with a plot ID; language and intent are detected.	Answer consistent with the dashboard
Accept / override advice	Farmer, Supervisor, Agronomist	Record whether advice was followed, the actual irrigation day and a reason.	Entry in feedback log for retraining
View region dashboard	Supervisor, Factory	KPIs, 14-day irrigation calendar, stress and risk distribution, taluk tables, priority plots.	Region-level situational awareness
Set weather scenario	Agronomist	Forecast rain, temperature and humidity anomaly, dry days since observation, method, pump flow.	All views recomputed
Optimise pump schedule	Supervisor, Factory	Select area, power windows, feeder capacity and horizon.	WSPT timetable with KPIs vs rotation

Use case	Actor(s)	Description	Outcome
Run what-if simulation	Agronomist	Change any input with sliders and observe all predictions.	Live predictions and drivers
Retrain ML models	Data Scientist	Run the training pipeline on new or real labelled data.	Updated models, metrics and fixtures

4.2.2 Data-flow diagrams

The Level-0 data-flow diagram (context diagram) in Fig. 4.2 treats SMA as a single process. Satellite/GIS sources supply plot features, the weather scenario supplies forecast rain and anomalies, farmers supply plot IDs, questions and feedback, and supervisors supply power windows, feeder capacity and overrides. SMA returns plot advisories to farmers, dashboards and pump timetables to supervisors, and water-demand calendars and yield estimates to the sugar factory.

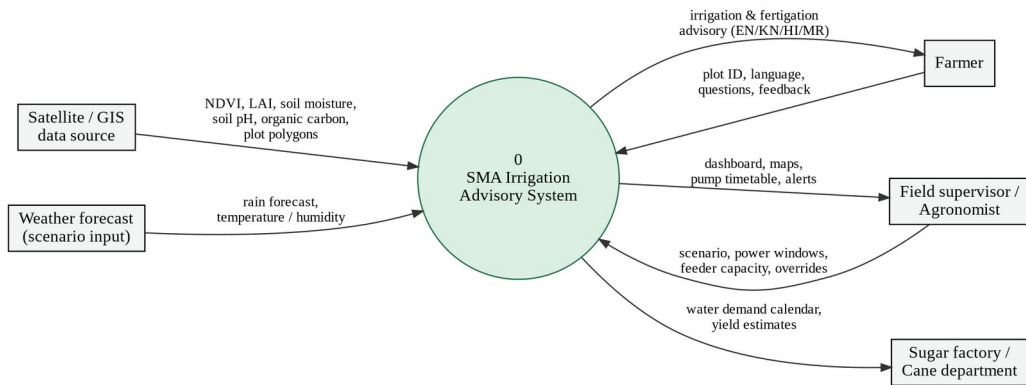


Fig. 4.2: Level-0 data-flow diagram (context diagram)

The Level-1 data-flow diagram in Fig. 4.3 decomposes SMA into seven processes and three data stores. Raw plot records are ingested and cleaned (1.0) into the plot dataset D1. The training process (2.0) reads D1 and writes the trained models D2. The prediction process (3.0) combines features from D1, models from D2 and the weather scenario. Predictions flow into the decision engine (4.0), which feeds both the pump scheduler (5.0) and the advisory generator (6.0). User decisions are recorded (7.0) in the feedback log D3, which closes the learning loop by providing retraining data to process 2.0.

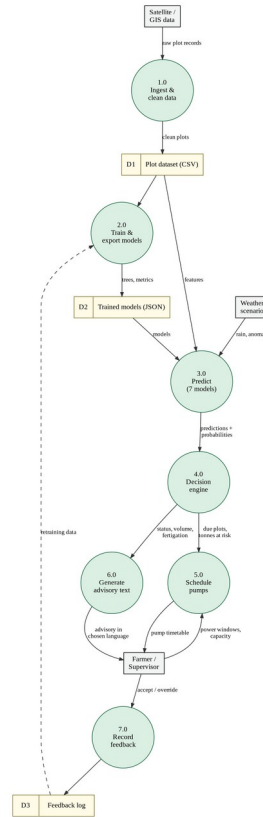


Fig. 4.3: Level-1 data-flow diagram

4.2.3 Class diagram

The class diagram in Fig. 4.4 shows the main software abstractions. A Farm holds the observed features, polygon and data-quality flags of a plot, and a Scenario holds the weather and operating parameters. The AdvisoryEngine is the central class: it builds model features from a Farm and a Scenario, obtains predictions from the ModelRegistry (which aggregates seven TreeEnsemble objects), and creates an Advisory with the help of the FertigationPlanner, the ExplainEngine and the I18n component. The PumpScheduler and the InsightsBuilder consume advisory summaries, and the ChatBot reuses the AdvisoryEngine and I18n so that chatbot answers are identical to the dashboard.

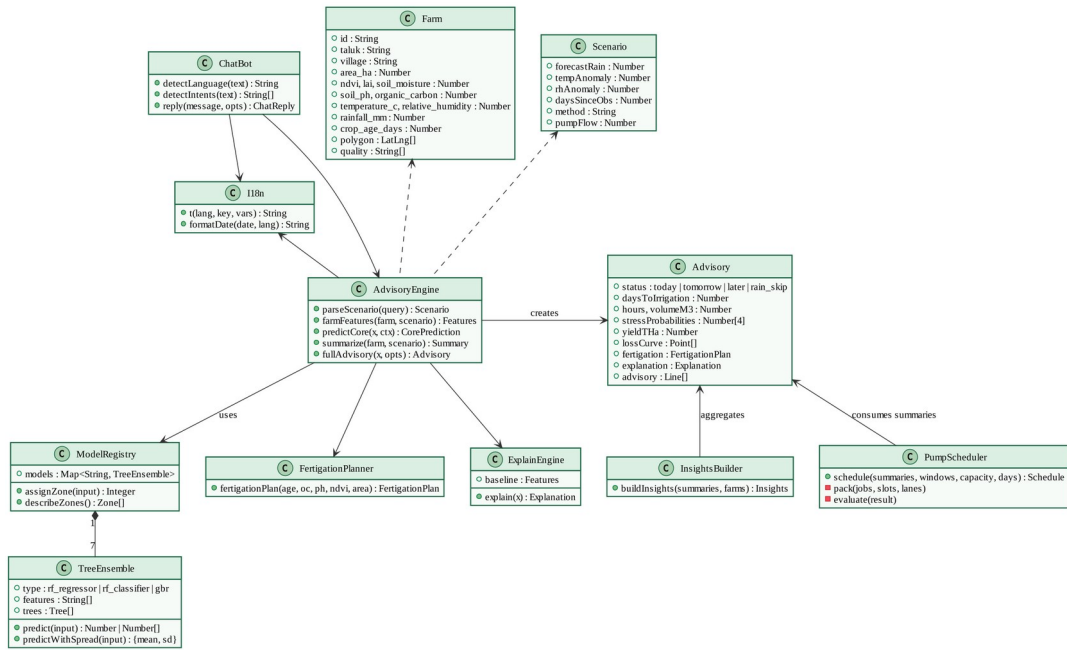


Fig. 4.4: Class diagram

4.2.4 Sequence diagrams

Fig. 4.5 shows the sequence for the most frequent interaction — a farmer requesting a plot advisory in Kannada. The web UI calls the REST API, which validates and clamps the scenario and invokes the AdvisoryEngine. The engine applies the weather scenario, queries all seven models, computes the rain counterfactual, hydraulics, fertigation plan and 14-day projection, asks the ExplainEngine for feature effects and asks I18n to render the text. The optional fragment shows the farmer overriding the advice, which is stored through the feedback endpoint.

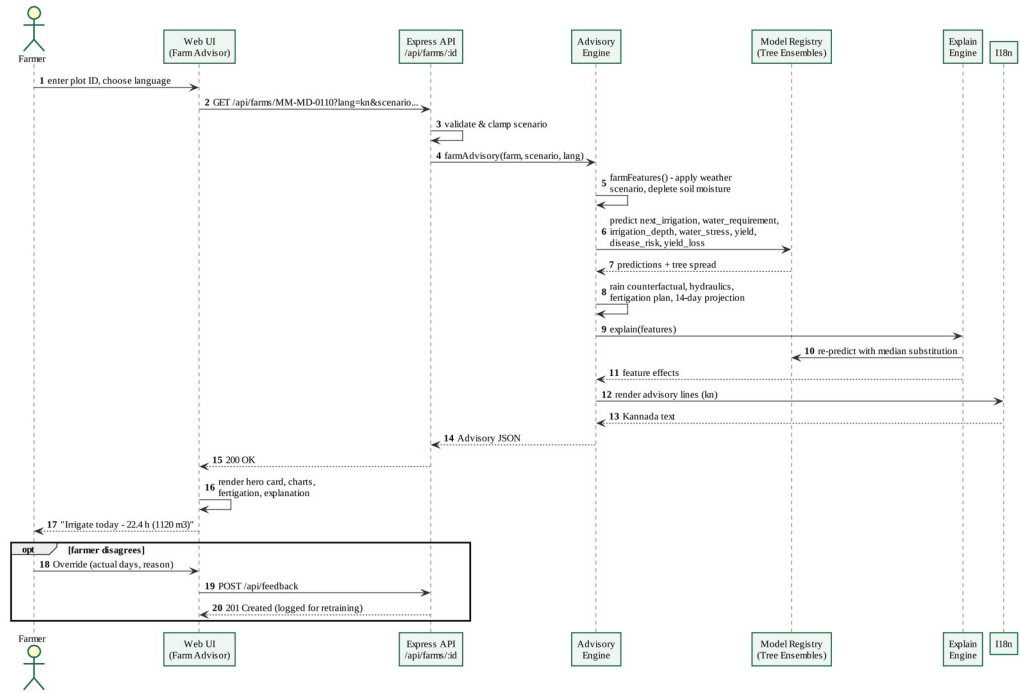


Fig. 4.5: Sequence diagram: plot advisory request

Fig. 4.6 shows how a supervisor obtains an optimised pump timetable. Summaries for the selected village are taken from the per-scenario cache; the scheduler builds the power slots, weights every plot by urgency and cane at risk, orders plots by the WSPT rule, packs them into feeder lanes and evaluates the plan against a first-come rotation.

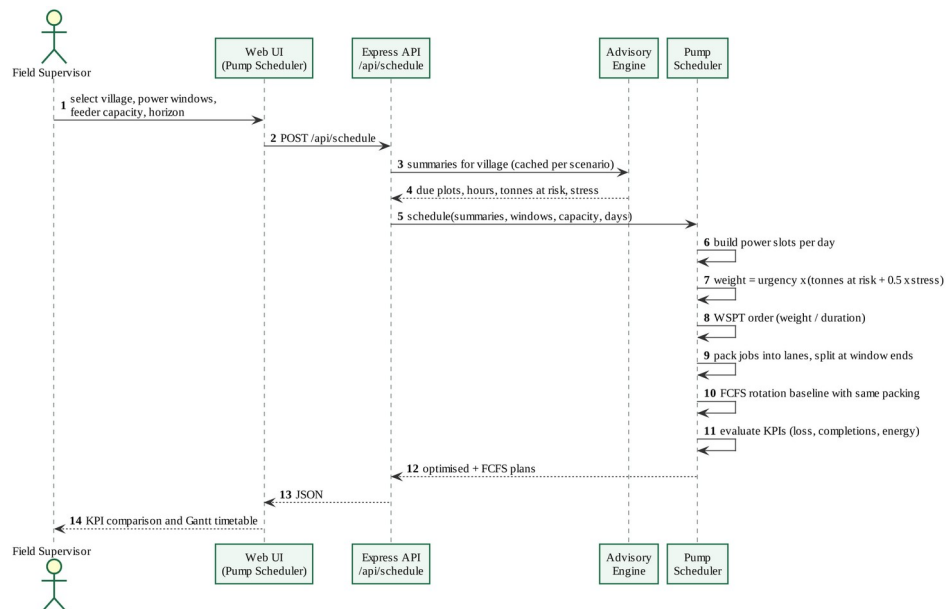


Fig. 4.6: Sequence diagram: pump schedule optimisation

4.2.5 State diagram

The state diagram in Fig. 4.7 models the irrigation life-cycle of a plot. A plot is Monitoring while the root zone holds more than the readily available water. As the crop

uses water it moves to Due Soon and then Due Today/Tomorrow. If forecast rain would refill the root zone, it enters Rain Skip and returns to monitoring once the rain arrives (or back to Due if the forecast fails). If irrigation is delayed beyond the readily available water, the plot becomes Water Stressed, which gives it high priority in the scheduler. Irrigation takes the plot to Refilled and then back to Monitoring until harvest.

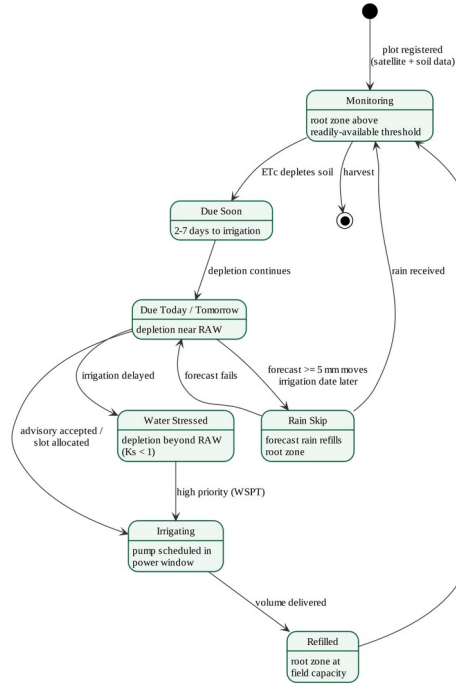


Fig. 4.7: State diagram of a plot's irrigation status

4.2.6 Activity diagrams

The activity diagram in Fig. 4.8 describes the offline training pipeline: loading and cleaning data, generating seasonal scenarios, labelling them with the agronomy engine, adding noise, splitting by plot, training the four groups of models in parallel, evaluating against baselines, clustering management zones, exporting to JSON and verifying parity with the Node.js tests before serving.

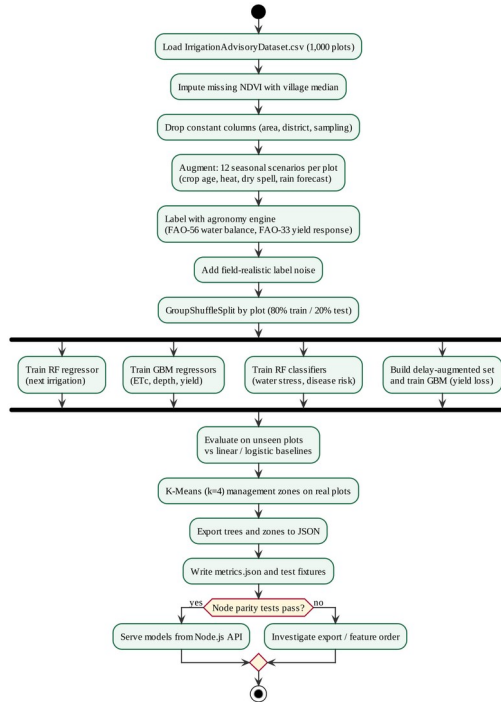


Fig. 4.8: Activity diagram: model training pipeline

The activity diagram in Fig. 4.9 describes the decision logic for one advisory, including soil depletion for dry days, the rain-skip rule, the irrigation status, hydraulics, fertigation, explanation and text generation.

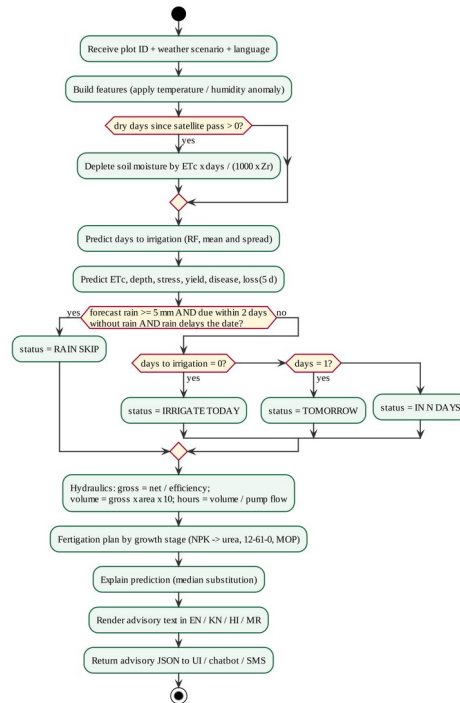


Fig. 4.9: Activity diagram: advisory generation

4.2.7 Component and deployment diagrams

The component diagram in Fig. 4.10 shows the three parts of the system: the offline Python training package, the Node.js server (REST API, data loader, tree-ensemble inference, advisory engine, scheduler, explanation, insights, internationalisation and chatbot) and the browser single-page application that uses Chart.js and Leaflet. Trained models and metrics are the only artefacts passed from training to serving.

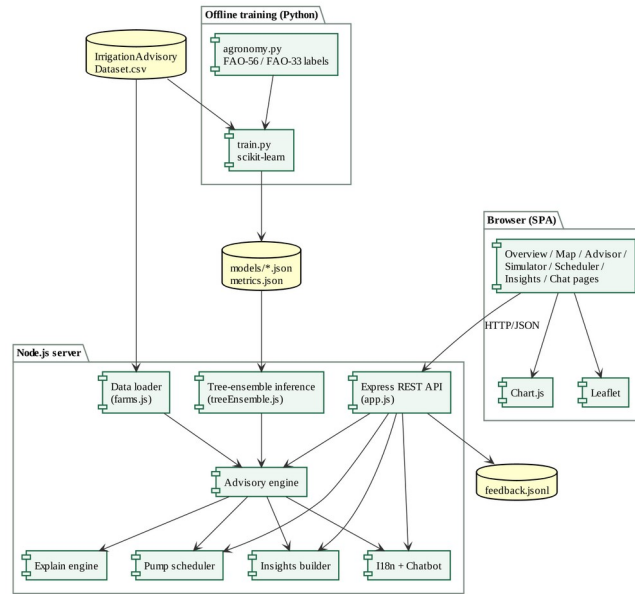


Fig. 4.10: Component diagram

The deployment diagram in Fig. 4.11 shows that the system needs only a user device with a browser and an application server with Node.js; the training workstation is used offline. Map tiles are fetched from public tile services, and integration with IMD forecasts, Sentinel-2 imagery and IoT gateways is planned.

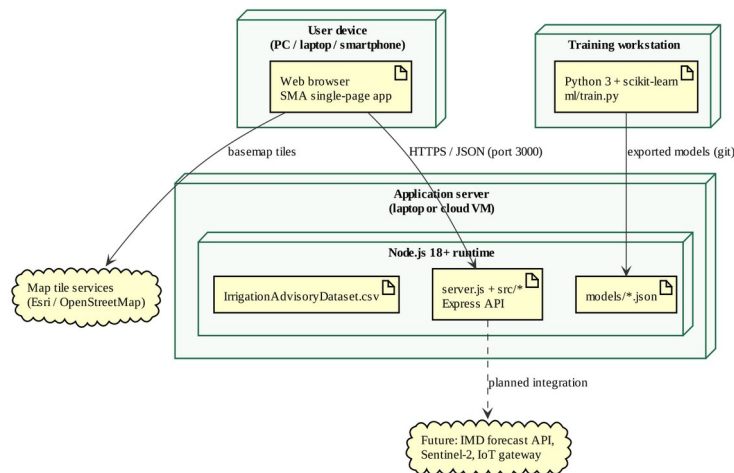


Fig. 4.11: Deployment diagram

4.3 System Architecture

SMA follows a layered architecture. The data layer consists of the plot dataset (CSV with GeoJSON polygons), the trained model files (JSON) and the feedback log (JSON Lines). The intelligence layer contains the tree-ensemble inference engine and the model registry. The decision layer contains the advisory engine, fertigation planner, explanation engine, pump scheduler and insights builder. The service layer is a stateless Express REST API that validates inputs and caches plot summaries per weather scenario. The presentation layer is a single-page web application with eight views. Because each layer depends only on the one below it, the ML models can be retrained, the data source replaced (for example by live STEPS records) or new front-ends (mobile app, SMS gateway, WhatsApp bot) added without changing the other layers.

Two architectural choices are worth highlighting. First, training and serving are separated: Python is used only offline, and the exported models are executed in JavaScript with a float32 comparison that reproduces scikit-learn exactly, so the deployed system has a single runtime. Second, all views share one prediction path: the dashboard, map, advisor, scheduler and chatbot all call the same advisory engine with the same scenario, so a farmer, a supervisor and the chatbot always see the same answer.

4.4 Data Design

The data dictionary of the input dataset is given in Table 4.2. Each record is one geofenced plot sampled on the ESA WorldCover cropland class [14].

Table 4.2: Data dictionary of the IrrigationAdvisoryDataset

Attribute	Type / unit	Description	Used as
Farm_ID	Text	Anonymised plot identifier (e.g. MM-MD-0110)	Key
District, Taluk, Village	Text	Administrative location (Mandya; 6 taluks; 10 villages)	Grouping, filters
Farm_Area_ha	Number, ha	Plot area (constant 0.7355 ha)	Volume computation
NDVI	Number, –	Normalized Difference Vegetation Index (6 missing values)	Model feature
LAI	Number, m ² /m ²	Leaf Area Index	Model feature
Soil_Moisture	Number, m ³ /m ³	Volumetric soil moisture	Model feature
Soil_pH	Number, –	Soil reaction	Model feature

Attribute	Type / unit	Description	Used as
Organic_Carbon	Number, g/kg	Soil organic carbon	Model feature
Rainfall_mm	Number, mm	Seasonal rainfall	Model feature
Temperature_C	Number, °C	Mean air temperature	Model feature
Relative_Humidity	Number, %	Mean relative humidity	Model feature
Sampling_Method	Text	ESA_WorldCover_Cropland (constant)	Metadata
.geo	GeoJSON	Plot boundary polygon	Map, centroid

In addition to the observed attributes, two derived inputs are used: the crop age in days after planting (not present in the dataset; a stable estimate per plot is used and can be overridden, while in production it comes from STEPS planting records) and the forecast rain for the next three days. Each exported model is stored as JSON containing the model type, the ordered feature list and, for every tree, parallel arrays of split features, thresholds, left and right child indices and leaf values; Gradient-Boosting models additionally store the initial prediction and the learning rate.

4.5 User Interface Design

The user interface serves two audiences — farmers who need one clear instruction, and supervisors and agronomists who need region-wide analysis — and follows these principles:

- **One action first.** The advisor opens with a colour-coded banner ("Irrigate today", "Rain expected – skip irrigation") with pump hours and water volume.
- **Scenario-driven views.** Weather presets and sliders recompute every page for planning and demonstration.
- **Local language and voice.** Advice in English, Kannada, Hindi and Marathi, read aloud or copied as an SMS.
- **Accessible visuals.** Colour-blind-safe palette; stress and risk shown with icon and label, not colour alone; light and dark themes.
- **Transparency.** Probabilities, uncertainty, feature effects, accuracy and data-quality flags are visible.
- **Responsive layout.** Works on office monitors and field smartphones.

CHAPTER 5

METHODOLOGY

5.1 Introduction

This chapter describes how SMA turns plot observations into recommendations: data analysis, pre-processing, physics-informed label generation, machine-learning models, the decision engine, pump scheduling and explanation, each with its algorithm steps and mathematical formulation.

5.2 System Block Diagram

The system block diagram in Fig. 5.1 organises SMA into five layers. The input layer receives satellite indices, soil properties, seasonal climate, plot polygons, the weather scenario and the crop age. The pre-processing layer imputes missing values, flags anomalous readings and builds the model features, including soil-moisture depletion for dry days. The machine-learning layer holds seven supervised tree-ensemble models and one unsupervised clustering model, all executed natively in Node.js. The decision-support engine converts the predictions into a rain-adjusted irrigation status, water volume and pump hours, a fertigation plan, an optimised pump timetable and an explanation. The output layer delivers the results through the web dashboard and farm map, the plot advisory in four languages, the Krishi Mitra chatbot, the pump timetable and a REST API for integration with STEPS or a mobile application.

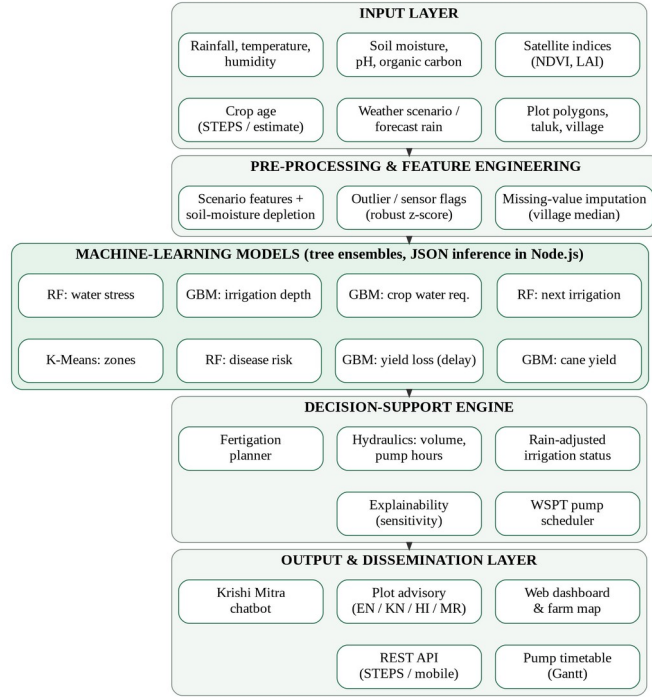


Fig. 5.1: System block diagram of SMA

5.3 Dataset Description and Exploratory Analysis

The IrrigationAdvisoryDataset contains 1,000 geofenced sugarcane plots in Mandya district, Karnataka, distributed over six taluks and ten villages (Fig. 5.2). Each plot has an area of 0.7355 ha, giving a total of 736 ha. Summary statistics of the numerical features are listed in Table 5.1.

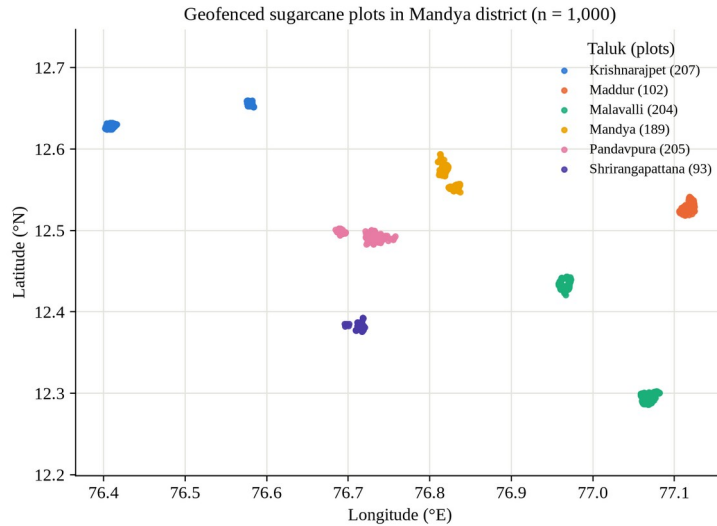


Fig. 5.2: Location of the 1,000 plots, coloured by taluk

Table 5.1: Descriptive statistics of the plot features

Feature	Mean	Std. dev.	Min	Max	Missing
NDVI	0.580	0.089	0.260	0.787	6
LAI (m ² /m ²)	0.602	0.090	0.400	0.850	0
Soil moisture (m ³ /m ³)	0.223	0.010	0.218	0.256	0
Soil pH	6.37	0.23	6.00	7.20	0
Organic carbon (g/kg)	10.54	1.85	7.67	20.00	0
Rainfall (mm)	629.6	74.6	498.3	711.3	0
Temperature (°C)	23.25	0.58	22.20	24.17	0
Relative humidity (%)	78.88	1.90	76.20	82.67	0

The feature distributions show that NDVI is approximately bell-shaped around 0.58, while soil moisture, temperature and humidity vary within narrow ranges because the satellite pass corresponds to a single, relatively moist season. Soil moisture ranges only from 0.218 to 0.256 m³/m³. The Pearson correlation matrix (also shown on the Model Insights page) reveals strong relationships between the climate variables: temperature is negatively correlated with humidity ($r = -0.94$) and rainfall ($r = -0.85$), and rainfall and humidity are positively correlated ($r = 0.85$). Such collinearity does not harm tree ensembles, but it matters for explanation, as discussed in Section 5.9.

Three conclusions shaped the methodology: (i) there are no target labels, so labels must be generated; (ii) the data cover one season, so scenario augmentation is needed to learn behaviour in dry, hot or rainy weather and at all crop ages; and (iii) area, district and sampling method are constant.

5.4 Data Pre-processing

Missing values. Six NDVI values are missing, most likely because of cloud masking. Because neighbouring plots share climate and management, each missing value is imputed with the median NDVI of the plot's village, and the imputation is recorded as a data-quality flag.

Outlier detection. Faulty or noisy sensor readings are a key operational challenge in the use case [2]. SMA flags unusual values with the modified z-score of Iglewicz and Hoaglin [26], computed within each village so that a genuinely wetter village is not mistaken for a sensor error:

$$z_i = (x_i - \tilde{x}) / (\text{MAD} / 0.6745), \quad \text{flag if } |z_i| > 3.5 \quad (5.1)$$

where $\tilde{x}$ is the village median and MAD is the median absolute deviation. When more than half of the values are identical (MAD = 0), the mean absolute deviation multiplied by 1.2533 is used as the scale. Flags are shown on the plot advisory so that field staff can verify the reading.

Constant columns (Farm_Area_ha, District, Sampling_Method) are excluded from the model inputs; the plot area is still used to convert depths into volumes.

Scenario features. At prediction time, the weather scenario modifies the observed features. The temperature anomaly ΔT_a is added to the temperature, humidity is reduced by 2.5 percentage points per degree of warming plus the humidity anomaly, and when n dry days have passed since the satellite observation the root zone is depleted with the predicted crop water use (FAO-56 water-balance bookkeeping):

$$\theta' = \max(\text{PWP}, \theta - \text{ET}_c \cdot n / (1000 \cdot Z_r)) \quad (5.2)$$

5.5 Physics-Informed Label Generation

Because the dataset contains no irrigation or yield records, SMA follows the theory-guided data science paradigm [21]: established agronomic models are used to generate labels, and machine-learning models are trained on them. The agronomy engine implements the FAO-56 crop water balance [6] and the FAO-33 yield response [11].

5.5.1 Scenario augmentation

Each of the 1,000 plots is expanded into twelve seasonal scenarios, giving 12,000 training rows. The sampling distributions are listed in Table 5.2. NDVI and LAI follow the expected sugarcane phenology curve scaled by the plot's observed vigour, so a plot that is greener than average remains greener in every scenario.

Table 5.2: Scenario augmentation parameters

Variable	Scenario sampling
Crop age a	Uniform 0–365 days after planting
Temperature T	Observed temperature + Uniform(−5, +13) °C
Relative humidity RH	Observed $RH - 2.5 \times \text{temperature anomaly} + \text{Normal}(0, 5)$, clipped to 25–98%
Soil moisture θ	30%: observed + Normal(0, 0.015); 70%: Uniform(0.10, 0.38) m^3/m^3
NDVI	Expected NDVI(a) $\times$ (observed NDVI / 0.58) + Normal(0, 0.04)
LAI	Observed LAI $\times$ Expected NDVI(a) / 0.6 + Normal(0, 0.05)
Forecast rain R	55%: 0 mm; 45%: Exponential(mean 15 mm), capped at 80 mm

Variable	Scenario sampling
Soil pH, organic carbon, rainfall	Observed value with small Gaussian perturbation

5.5.2 Crop water requirement

Reference evapotranspiration is estimated with the Hargreaves–Samani equation [10], where $R_a = 14.5$ mm/day is the mean extraterrestrial radiation for latitude $\approx 12.5^\circ$ N and the diurnal temperature range ΔT is estimated from humidity:

$$ET_0 = 0.0023 \cdot R_a \cdot (T + 17.8) \cdot \sqrt{\Delta T}, \quad \Delta T = \text{clip}(16 - 0.12 \cdot RH, 4, 14) \quad (5.3)$$

The crop coefficient blends the FAO-56 stage curve for a twelve-month sugarcane crop (Table 5.3) with an NDVI-based coefficient, so that a vigorous canopy transpires more:

$$K_c = 0.6 \cdot K_{c,\text{stage}}(a) + 0.4 \cdot \text{clip}(1.5 \cdot \text{NDVI} + 0.05, 0.35, 1.30) \quad (5.4)$$

$$ET_c = K_c \cdot ET_0 \quad (5.5)$$

Table 5.3: Sugarcane crop-coefficient curve and stage sensitivity

Growth stage	Crop age (days)	$K_{c,\text{stage}}$	Stage sensitivity w
Germination	0 – 45	0.45	0.8
Tillering	45 – 130	0.45 → 1.25 (linear)	1.2
Grand growth	130 – 290	1.25	1.5
Maturity	290 – 365	1.25 → 0.75 (linear)	0.5

The resulting crop-coefficient curve and the expected NDVI and root-depth trajectories are plotted in Fig. 5.3.

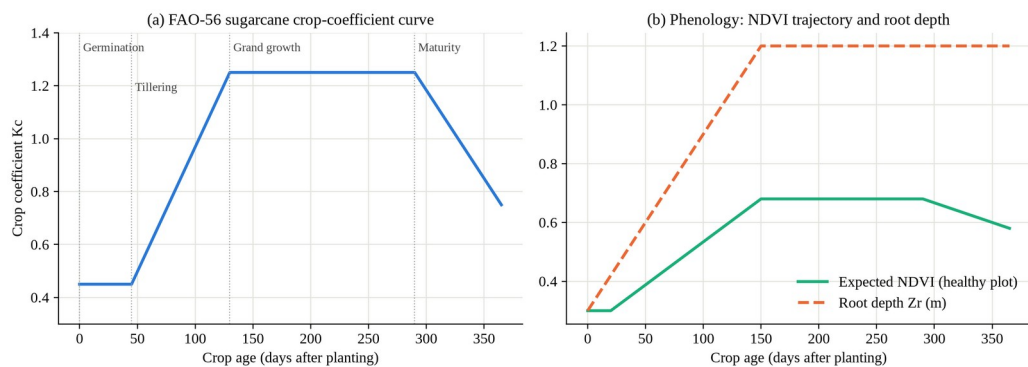


Fig. 5.3: FAO-56 sugarcane crop coefficient, expected NDVI and root depth versus crop age

5.5.3 Root-zone water balance and irrigation timing

Soil water-holding capacity increases with organic carbon (OC, g/kg), and the root depth grows from 0.3 m to 1.2 m during the first 150 days:

$$FC = 0.27 + 0.003 \cdot OC, \quad PWP = 0.12 + 0.0015 \cdot OC \quad (5.6)$$

$$Z_r = 0.3 + 0.9 \cdot \min(a, 150) / 150 \quad (5.7)$$

The total and readily available water and the current root-zone depletion D_r follow FAO-56, with the depletion fraction p adjusted for evaporative demand:

$$TAW = 1000 \cdot (FC - PWP) \cdot Z_r, \quad D_r = \text{clip}(1000 \cdot (FC - \theta) \cdot Z_r, 0, TAW) \quad (5.8)$$

$$p = \text{clip}(0.65 + 0.04 \cdot (5 - ET_c), 0.4, 0.8), \quad RAW = p \cdot TAW \quad (5.9)$$

Irrigation is due when the depletion reaches RAW. Effective forecast rain P_{eff} delays that moment, giving the label for the number of days to irrigation and the net irrigation depth needed to refill the root zone:

$$P_{eff} = 0.8 \cdot \max(0, R - 3), \quad t_{irr} = \text{clip}((RAW - D_r + P_{eff}) / ET_c, 0, 15) \quad (5.10)$$

$$d_{net} = RAW \text{ if } t_{irr} > 0, \text{ otherwise } \max(0, D_r - P_{eff}) \text{ (capped at 100 mm)} \quad (5.11)$$

The water balance is illustrated in Fig. 5.4: without irrigation the depletion crosses RAW (stress begins) and approaches TAW (wilting); SMA recommends irrigation when RAW is reached, refilling the root zone to field capacity.

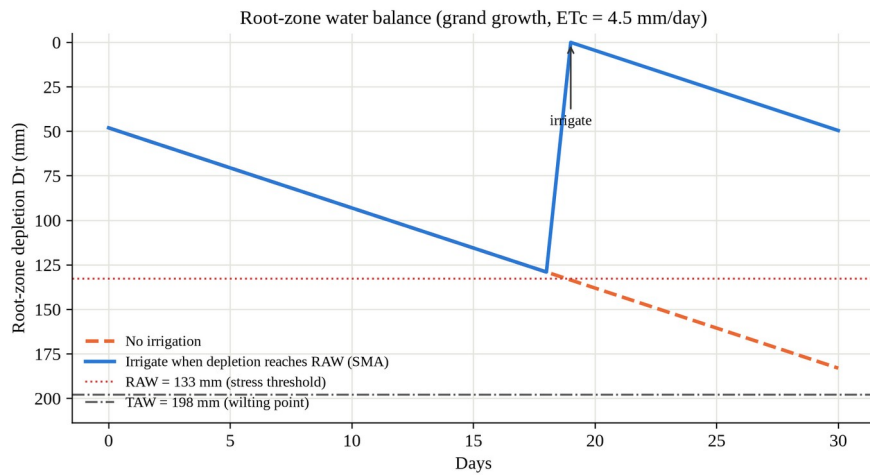


Fig. 5.4: Root-zone depletion with and without SMA-scheduled irrigation

5.5.4 Water stress, yield, yield loss and disease risk

The FAO-56 water-stress coefficient K_s becomes less than one once depletion exceeds RAW. The water-stress score combines K_s with heat stress h , NDVI deficit n and

closeness to the RAW threshold q , and is discretised into four classes (None < 0.10 ≤ Mild < 0.30 ≤ Moderate < 0.55 ≤ Severe):

$$K_s = (TAW - D_r) / ((1 - p) \cdot TAW) \text{ if } D_r > RAW, \text{ else } 1 \quad (5.12)$$

$$s = \text{clip}(0.7 \cdot (1 - K_s) + 0.15 \cdot h + 0.15 \cdot n + 0.15 \cdot q, 0, 1) \quad (5.13)$$

where $h = \text{clip}((T - 33)/7, 0, 1)$, $n = \text{clip}((NDVI_{\text{exp}}(a) - NDVI)/0.3, 0, 1)$ and $q = \text{clip}((D_r/RAW - 0.7)/0.3, 0, 1)$. Attainable yield ($Y_{\text{max}} = 125$ t/ha for the region) is reduced by multiplicative factors for soil pH, organic carbon, vigour, temperature, rainfall and current stress:

$$Y = Y_{\text{max}} \cdot g_{\text{pH}} \cdot g_{\text{OC}} \cdot g_v \cdot g_T \cdot g_R \cdot (1 - 0.25 \cdot s) \quad (5.14)$$

The yield loss caused by delaying irrigation by d days is obtained by simulating the daily depletion over the delay and integrating the FAO-33 relationship with the sugarcane yield-response factor $K_y = 1.2$, the stage sensitivity w from Table 5.3 and a heat multiplier η_h :

$$1 - Y_d/Y_m = K_y \cdot (1 - ET_d/ET_m) \quad (5.15)$$

$$L(d) = 100 \cdot K_y \cdot [\sum_{t=1..d} (1 - K_{s,t}) / 365] \cdot w \cdot 2 \cdot \eta_h \quad (\%) \quad (5.16)$$

Disease risk takes the larger of a fungal score (humidity near a 27 °C optimum, wet soil, rain) and an early-shoot-borer score (hot, dry, young crop), discretised into Low, Medium and High. Every label receives field-realistic noise (e.g. ±0.6 days, ±5% ETc, ±4 t/ha yield) so that models learn patterns rather than memorising formulas.

5.6 Machine-Learning Models

Tree ensembles are used because they are the most accurate models for medium-sized tabular data [18], capture thresholds such as the RAW limit, give feature importance and export as plain data. The ten inputs are NDVI, LAI, soil moisture, pH, organic carbon, temperature, humidity, seasonal rainfall, crop age and forecast rain (plus delay days for the yield-loss model).

Random Forest [16] averages B decision trees, each grown on a bootstrap sample with random feature subsets. For regression the prediction and its spread across trees, used by SMA as an uncertainty estimate, are:

$$\hat{y}(x) = (1/B) \sum_{b=1..B} T_b(x), \quad \sigma(x) = \sqrt{[(1/B) \sum_b (T_b(x) - \hat{y}(x))^2]} \quad (5.17)$$

For classification, the class probabilities are the average of the leaf class proportions of all trees. Gradient Boosting [17] builds an additive model in which each new tree h_m fits the negative gradient (the residuals for squared loss) of the current model, scaled by the learning rate v :

$$F_M(x) = F_0 + v \cdot \sum_{m=1..M} h_m(x) \quad (5.18)$$

K-Means [27] groups the standardised observed features of the 1,000 real plots into $k = 4$ management zones by minimising the within-cluster sum of squares:

$$J = \sum_{k=1..K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \quad (5.19)$$

The configuration of every model is given in Table 5.4. Hyperparameters were chosen to balance accuracy with the size of the exported JSON files.

Table 5.4: Model configuration and hyperparameters

Model	Target	Algorithm	Hyperparameters
next_irrigation	Days to irrigation (0–15)	RF regressor	40 trees, max depth 12, min samples per leaf 8
water_requirement	ETc (mm/day)	GBM regressor	200 trees, depth 4, learning rate 0.05
irrigation_depth	Net depth (mm)	GBM regressor	250 trees, depth 4, learning rate 0.05
yield	Cane yield (t/ha)	GBM regressor	250 trees, depth 4, learning rate 0.05
yield_loss	Loss for a delay (%)	GBM regressor	300 trees, depth 5, learning rate 0.05; 36,000 rows (3 delays per scenario)
water_stress	4 stress classes	RF classifier	40 trees, max depth 12, min samples per leaf 5
disease_risk	3 risk classes	RF classifier	30 trees, max depth 10, min samples per leaf 5
zones	Management zone	K-Means	$k = 4$, 20 initialisations, standardised features

5.6.1 Validation strategy and metrics

Because each plot contributes twelve scenarios, a random split would leak information from a plot into both training and test sets. SMA therefore uses a group split by plot: 80% of the plots (9,600 rows) are used for training and the remaining 20% (2,400 rows) are never seen during training. Each model is compared with a baseline (linear regression or logistic regression). Regression models are evaluated with the coefficient of determination and the mean absolute error, and classifiers with accuracy and the macro-averaged F1 score [20]:

$$R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2, \quad MAE = (1/N) \Sigma |y_i - \hat{y}_i| \quad (5.20)$$

$$F1_{macro} = (1/C) \Sigma_c 2 \cdot P_c \cdot R_c / (P_c + R_c) \quad (5.21)$$

5.6.2 Ordinal decision rule

For ordered classes (None < Mild < Moderate < Severe), reporting the class with the highest probability can understate risk: probabilities of 0.03, 0.45, 0.41 and 0.11 give Mild by arg-max although there is a 52% chance of moderate-or-worse stress. SMA therefore reports the ordinal median — the first class at which the cumulative probability reaches 50% — which is Moderate in this example and is safer for farmers:

$$\hat{c} = \min\{ k : \Sigma_{j \leq k} P(j) \geq 0.5 \} \quad (5.22)$$

5.7 Decision-Support Engine

The decision engine turns model outputs into concrete actions. For each plot it performs the steps of Algorithm 5.1.

Algorithm 5.1: Rain-adjusted plot advisory

- Step 1: Build the feature vector from the plot and the weather scenario; deplete soil moisture for dry days using Eq. (5.2).
- Step 2: Predict days to irrigation t (mean and spread), ET_c, net depth, stress probabilities, yield, disease risk and loss for a 5-day delay.
- Step 3: If forecast rain $R > 0$, predict t_0 with $R = 0$ (no-rain counterfactual).
- Step 4: If $R \geq 5$ mm, $\text{round}(t_0) \leq 2$ and $\text{round}(t) > \text{round}(t_0)$, set status = RAIN SKIP; otherwise set status = TODAY, TOMORROW or IN N DAYS from $\text{round}(t)$.
- Step 5: Compute gross depth, water volume and pump hours for the irrigation method and pump flow; for drip irrigation also compute the daily run-time.
- Step 6: Compute the weekly water requirement and the saving against fixed-interval practice.
- Step 7: Generate the fertigation plan for the crop stage, the 14-day moisture projection, the yield-loss curve for 0–14 days and the explanation.
- Step 8: Render the advisory text in the requested language and return the result.

The irrigation hydraulics use the application efficiency η of the method (flood 0.55, furrow 0.65, sprinkler 0.75, drip 0.90), the plot area A in hectares and the pump flow Q in m³/h:

$$d_{\text{gross}} = d_{\text{net}} / \eta, \quad V = d_{\text{gross}} \cdot A \cdot 10 \text{ (m}^3\text{)}, \quad H = V / Q \text{ (hours)} \quad (5.23)$$

The water saving is measured against a conventional fixed schedule of 70 mm gross every 8 days, which is typical of farmer practice:

$$W_{\text{SMA}} = \max(0, 7 \cdot ET_c - P_{\text{eff}}) / \eta, \quad W_{\text{conv}} = 70 \times 7 / 8, \quad \text{Saving} = 1 - W_{\text{SMA}} / W_{\text{conv}} \quad (5.24)$$

Energy saved is the pumping time avoided times the power of a 5 HP (3.73 kW) motor. The fertigation planner splits a 250:100:125 kg N:P₂O₅:K₂O per hectare package by stage (Table 5.5), adjusts nitrogen for organic carbon (−10% above 7.5 g/kg, +25% below 5 g/kg), converts nutrients to 12-61-0, urea and muriate of potash, and adds notes on soil pH, vigour, stress and the no-nitrogen-after-eight-months rule.

Table 5.5: Stage-wise fertigation split of the NPK package

Stage (days)	N share	P ₂ O ₅ share	K ₂ O share
Germination (0–45)	10%	25%	10%
Tillering (45–130)	40%	45%	30%
Grand growth, early (130–240)	50%	30%	45%
Grand growth, late (240–290)	0%	0%	15%
Maturity (290+)	0%	0%	0%

5.8 Pump Scheduling Algorithm

Rural pump sets receive three-phase power only in fixed windows (about seven hours a day), and a feeder can run only a limited number of pumps at once. Each plot j that needs water within the horizon is a job with processing time p_j (pump hours) and weight w_j , derived from the cane at risk L_j predicted by the yield-loss model, the stress probability P_j and an urgency factor u_j (2 if due today, otherwise 1):

$$w_j = u_j \cdot (L_j + 0.5 \cdot P_j) + 0.01, \quad \text{objective: minimise } \sum_j w_j \cdot C_j \quad (5.25)$$

where C_j is the completion time of job j . For a single machine, Smith's rule [24] — processing jobs in decreasing order of w_j/p_j (Weighted Shortest Processing Time) — is optimal. For parallel feeder lanes with power windows, SMA uses WSPT list scheduling [25], as given in Algorithm 5.2.

Algorithm 5.2: WSPT pump scheduling under power windows

Step 1: Build the list of power slots for each day of the horizon from the configured windows (for example 06:00–10:00 and 17:00–20:00).

Step 2: Select plots due within the horizon (all plots for drip irrigation) that are not in rain-skip state; compute p_j and w_j .

Step 3: Sort jobs in decreasing order of w_j / p_j .

Step 4: For each job, choose the feeder lane that becomes free earliest; allocate minutes in the current power slot and continue in the next slot when power goes off, until the job is complete or the horizon ends.

Step 5: Repeat the packing with first-come-first-served (rotation) order as a baseline.

Step 6: Evaluate both plans: plots completed, cane at risk still pending, estimated cane loss, average finish time of high-stress plots, pump hours and energy.

Sorting dominates the running time, so the algorithm runs in $O(n \log n + n \cdot s)$ for n jobs and s power slots — milliseconds for a village.

5.9 Explainability Method

For each prediction, SMA computes a local explanation by replacing one feature at a time with the regional median $\tilde{x}_f$ and measuring the change in the prediction. A positive effect on days-to-irrigation means the feature delays irrigation compared with a typical plot:

$$\phi_f(x) = f(x) - f(x \text{ with } x_f := \tilde{x}_f) \quad (5.26)$$

This model-agnostic sensitivity analysis is related to SHAP [22] but cheap enough for every request; the three largest effects become sentences such as "Soil moisture is lower than the regional median, which brings forward irrigation by 4.8 days."

5.10 Multilingual Advisory and Chatbot

Predictions are converted into template-based sentences in English, Kannada, Hindi and Marathi, which keeps every sentence auditable and consistent with the dashboard; the same structured output can later feed a large language model, as the use case suggests.

The Krishi Mitra chatbot reuses the same engine: it detects the language from the Unicode script, detects the intent from multilingual keywords, extracts the plot ID and returns the relevant advisory lines, so it never contradicts the dashboard (implementation details in Section 6.7).

CHAPTER 6

IMPLEMENTATION DETAILS

6.1 Introduction

SMA is implemented as an offline Python pipeline (about 470 lines) that labels, trains, evaluates and exports the models, and a Node.js application (about 3,100 lines) that serves the REST API and a single-page web application. This chapter presents the user interface, project structure, training and inference, API, security, testing and deployment.

6.2 System Implementation

The web application has eight views reachable from a sidebar: Overview, Farm Map, Farm Advisor, What-If Simulator, Pump Scheduler, Model Insights, Krishi Mitra and About & Governance. A weather-scenario bar at the top of the operational views provides four presets and sliders for forecast rain, temperature and humidity anomalies, dry days since the satellite pass, irrigation method and pump flow. The screenshots below were captured with the Summer dry spell preset (seven dry days since the satellite observation and a temperature 8 °C above normal) unless stated otherwise.

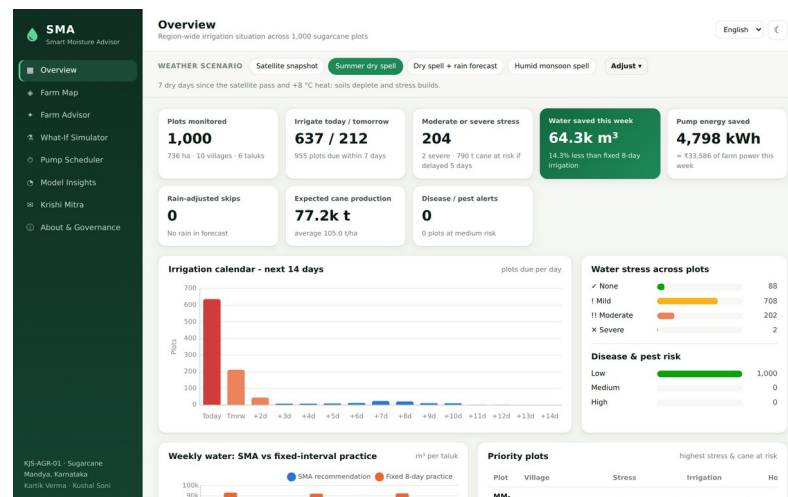


Fig. 6.1: Overview dashboard (summer dry spell scenario)

Overview dashboard (Fig. 6.1). Key indicators show the plots monitored (1,000 plots, 736 ha), plots to irrigate today and tomorrow (637 and 212), plots under moderate or severe stress (204), weekly water and pump energy saved against fixed eight-day irrigation, rain-adjusted skips, expected cane production and disease alerts. A 14-day

irrigation calendar, stress and risk bars, taluk-wise water comparison, a Priority plots table (each row opens the plot advisory) and a taluk/village summary table follow.

Farm map (Fig. 6.2). The Leaflet map shows every plot coloured by a selectable layer (irrigation due date, water stress, NDVI, soil moisture, management zone or disease risk) with a legend, a taluk filter and village labels that keep it readable offline. Zooming in draws the real plot polygons, and clicking a plot opens a side panel with its status, key numbers, data-quality flags and a link to the full advisory. The figure shows Maddur taluk coloured by water stress.

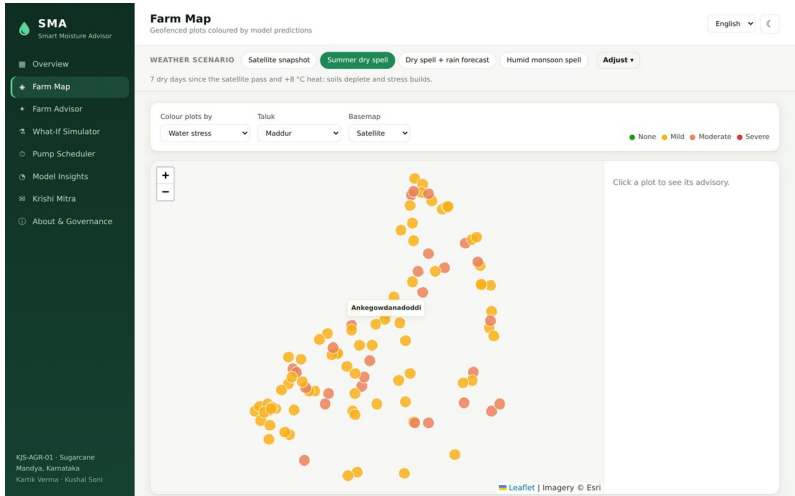


Fig. 6.2: Farm map filtered to Maddur taluk, coloured by water stress

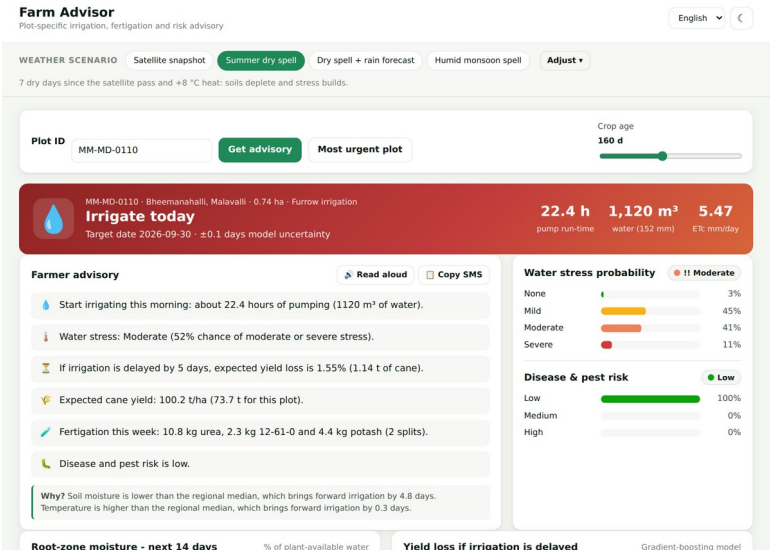


Fig. 6.3: Farm Advisor: action banner, farmer advisory and probabilities for plot MM-MD-0110

Farm Advisor (Fig. 6.3). A colour-coded action banner (red for Irrigate today, amber for soon, blue for Rain expected – skip) shows the plot, the target date with model uncertainty (± 0.1 days), pump run-time (22.4 h), water volume (1,120 m³) and crop

water requirement (5.47 mm/day). The Farmer advisory card gives six plain-language lines (irrigation, stress, cost of a five-day delay, yield, fertigation, disease) and a Why? sentence, with Read aloud and Copy SMS buttons. The probability card shows the full stress distribution (3%, 45%, 41%, 11%), for which the ordinal rule reports Moderate stress.

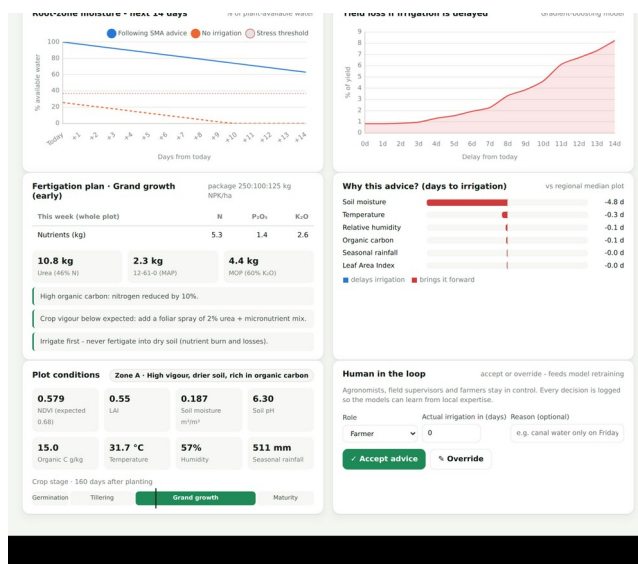


Fig. 6.4: Farm Advisor: moisture projection, yield-loss curve, fertigation, explanation and feedback

Advisor analytics (Fig. 6.4). The lower part shows the 14-day root-zone projection with and without irrigation, the yield-loss curve for delays of 0–14 days, the weekly fertigation plan in nutrients and products, the Why this advice? feature effects (soil moisture brings irrigation forward by 4.8 days), the plot conditions with zone and crop stage, and the human-in-the-loop accept/override controls.

Multilingual advisory. Selecting ಕನ್ನಡ, हिंदी or मराठी regenerates the advisory in that language with local date formats; an English translation remains available for verification (examples in Table 6.3).

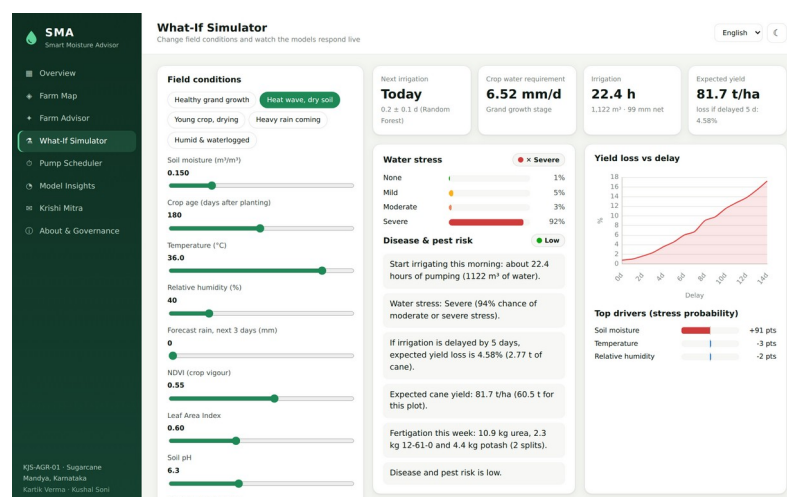


Fig. 6.5: What-If Simulator with the "Heat wave, dry soil" preset

What-If Simulator (Fig. 6.5). Sliders for all ten inputs, the irrigation method and the plot area, plus presets such as Heat wave, dry soil and Heavy rain coming, update the predictions almost instantly. In the heat-wave example the models predict irrigation today, 6.52 mm/day of crop water use, a 92% probability of severe stress and a 4.58% yield loss for a five-day delay, driven mainly by soil moisture.

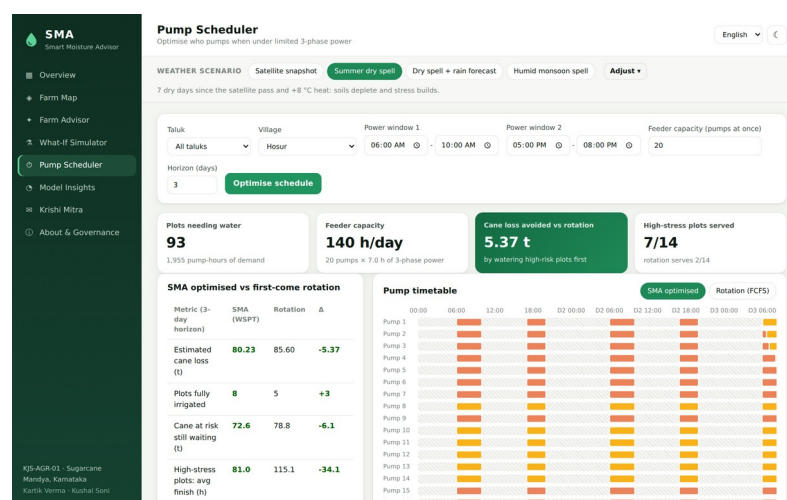


Fig. 6.6: Pump Scheduler for village Hosur with two power windows and 20 pumps per feeder

Pump Scheduler (Fig. 6.6). The supervisor chooses the area, two power windows, feeder capacity and horizon. KPI cards compare demand with capacity and show the cane loss avoided and high-stress plots served versus a first-come rotation; a Gantt chart shows each pump lane with hatched no-power periods and segments coloured by stress, and can be toggled between the SMA plan and the rotation.

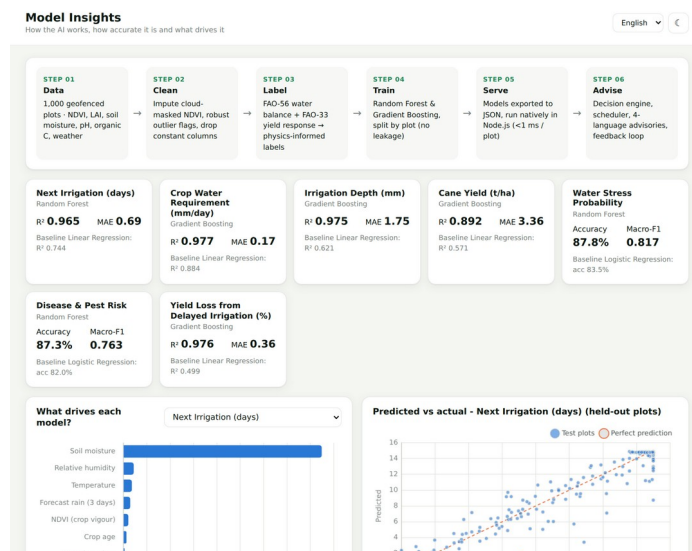


Fig. 6.7: Model Insights: pipeline, model accuracy cards, feature importance and predicted-versus-actual plot

Model Insights (Fig. 6.7). The pipeline, test metric and baseline of every model, feature importance, predicted-versus-actual plots or confusion matrices, feature correlations, management zones, NDVI distribution, data-quality statistics and the feedback acceptance rate make the AI transparent.

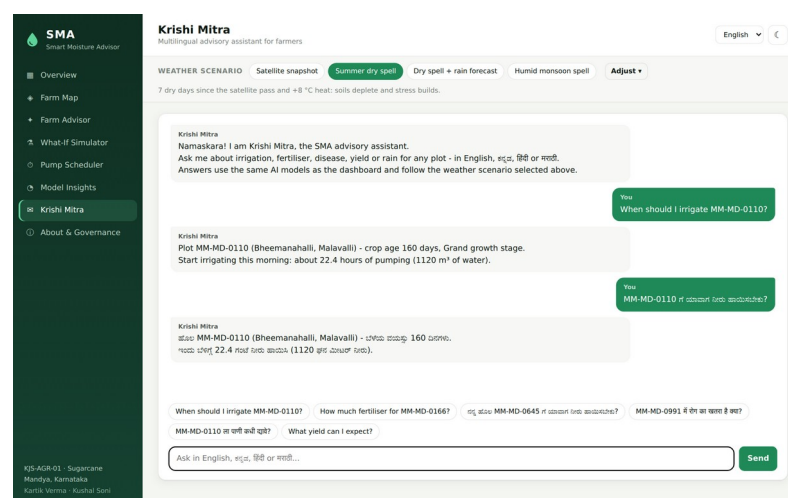


Fig. 6.8: Krishi Mitra chatbot answering in English and Kannada

Krishi Mitra (Fig. 6.8). The chatbot answers questions in any of the four languages; the same question about plot MM-MD-0110 asked in English and in Kannada returns the plot summary and irrigation instruction in the language of the question. The About & Governance view maps the eleven required models to their implementation and lists the responsible-AI measures.

6.3 Project Structure and Modules

The code is organised into the modules in Table 6.1; the client uses native ES modules without a build step.

Table 6.1: Project modules

Path	Module	Responsibility
ml/agronomy.py	Agronomy engine	FAO-56 / FAO-33 functions used to generate labels
ml/train.py	Training pipeline	Augmentation, labelling, training, evaluation, export, fixtures
models/*.json	Trained models	Seven tree ensembles, K-Means zones, metrics.json, test fixtures
src/data/farms.js	Data loader	CSV parsing, NDVI imputation, outlier flags, polygons, crop age
src/ml/treeEnsemble.js	Inference engine	RF / GBM tree walking with float32 parity, uncertainty
src/ml/registry.js	Model registry	Loads models, assigns management zones
src/engine/advisory.js	Advisory engine	Scenario features, predictions, rain rule, hydraulics, advisory
src/engine/agronomy.js	Agronomy helpers	Water-balance projection, growth stage, fertigation planner
src/engine/scheduler.js	Pump scheduler	Power slots, WSPT packing, rotation baseline, KPIs
src/engine/explain.js	Explain engine	Median-substitution feature effects and reasons
src/engine/insights.js	Insights builder	KPIs, calendar, taluk / village statistics, correlations
src/engine/i18n.js, chat.js	Language layer	EN / KN / HI / MR templates, chatbot
src/app.js, server.js	REST API	Routes, validation, caching, static hosting
public/	Web application	HTML, CSS and eight page modules (Chart.js, Leaflet)
test/	Test suite	Model parity, engine and API tests (node:test)

6.4 Training Pipeline Implementation

The training script loads the dataset with pandas, builds the scenario table, labels it with the agronomy engine and trains each model with scikit-learn [19]. Every fitted tree is exported as five parallel arrays — split feature (−1 for a leaf), threshold, left child, right child and leaf value — which keeps the JSON compact and makes inference a simple loop. Thresholds are written with nine significant digits so that the float32 comparison used by scikit-learn is reproduced exactly.

The script also writes metrics.json (test scores, baselines, feature importance, confusion matrices and prediction samples) and test_fixtures.json (40 held-out rows with the Python predictions), which the Node.js tests use to verify that inference matches.

6.5 Tree-Ensemble Inference in Node.js

The inference engine walks each tree from the root to a leaf. scikit-learn casts inputs to 32-bit floats before comparing them with thresholds; the engine reproduces this with Math.fround, which is why the JavaScript predictions are identical to Python. Random-Forest regressors average the leaves (and return their standard deviation as uncertainty), Random-Forest classifiers average class-probability vectors, and Gradient-Boosting models add the scaled sum of the leaves to the initial value.

```
function leafValue(tree, x) {
  let i = 0;
  while (tree.f[i] !== -1) {
    i = Math.fround(x[tree.f[i]]) <= tree.t[i] ? tree.l[i] : tree.r[i];
  }
  return tree.v[i];
}
// Gradient Boosting: init + learningRate * sum of leaf values
// Random Forest:    mean of leaf values (or of class-probability
//                  vectors)
```

Listing 6.1: Tree traversal with float32 parity (src/ml/treeEnsemble.js)

Plot summaries for all 1,000 plots are computed once per weather scenario and cached (up to 30 scenarios), so switching pages is instantaneous. A full region summary takes about 0.2 s and a detailed single-plot advisory, including the 15-point yield-loss curve and the explanation, about 3 ms.

6.6 REST API

All functionality is exposed through a stateless JSON REST API (Table 6.2) that STEPS, a mobile app or an SMS gateway can reuse; inputs are validated (HTTP 400 for malformed input, 404 for unknown plots).

Table 6.2: REST API endpoints

Method	Endpoint	Description
GET	/api/health	Health check (plots and models loaded)
GET	/api/meta	Taluks and villages, languages, methods, defaults, bounds
GET	/api/farms?taluk=&village=	Summary advisory for every plot under a scenario
GET	/api/farms/geo	Plot polygons for the map

Method	Endpoint	Description
GET	/api/farms/:id?lang=kn&cropAge=	Full advisory for one plot in a language
POST	/api/predict	What-if prediction for arbitrary inputs
GET	/api/insights	Region KPIs, calendar, taluk / village tables, correlations, zones
POST	/api/schedule	Optimised pump schedule and rotation baseline
GET	/api/models	Model metrics, importance, confusion matrices, sizes
POST / GET	/api/feedback	Record and summarise accept / override decisions
POST	/api/chat	Krishi Mitra chatbot

6.7 Multilingual Advisory and Chatbot Implementation

The language layer stores one template per message type and language (for example irrigate_today, rain_skip, stress, delay, yield, fert and fungal), with placeholders such as {hours}, {volume} and {date}. The advisory engine fills the placeholders with model outputs and formats dates with the locale of the chosen language (en-IN, kn-IN, hi-IN, mr-IN), so that the same numbers appear in every language. Table 6.3 shows the first two lines of the advisory for plot MM-MD-0110 when 40 mm of rain is forecast during a dry spell. Kannada and Devanagari text is rendered with Unicode fonts, and the browser speech-synthesis API reads it aloud when a voice for the language is installed.

Table 6.3: The same advisory generated in four languages (plot MM-MD-0110, rain forecast)

Language	Advisory text generated by SMA
English	Rain is expected (40 mm in 3 days) - do not irrigate now. Next irrigation around Sun, 4 Oct. Water stress: Mild (49% chance of moderate or severe stress).
Kannada	ಮಳೆ ನಿರೀಕ್ಷಿಸಲಾಗಿದೆ (3 ದಿನಗಳಲ್ಲಿ 40 ಮಿ.ಮೀ) - ಈಗ ನೀರು ಹಾಯಿಸಬೇಡಿ. ಮುಂದಿನ ನೀರಾವರಿ ಸುಮಾರು ಭಾನು, 4 ಅಕ್ಟೋಬರ್ ರಂದು. ನೀರಿನ ಒತ್ತಡ: ಸ್ವಲ್ಪ (ಮಧ್ಯಮ ಅಥವಾ ತೀವ್ರ ಒತ್ತಡದ ಸಾಧ್ಯತೆ 49%).
Hindi	बारिश की संभावना है (3 दिनों में 40 मिमी) - अभी सिंचाई न करें। अगली सिंचाई लगभग रवि, 4 अक्टूबर को। जल तनाव: हल्का (मध्यम या गंभीर तनाव की संभावना 49%)।
Marathi	पावसाची शक्यता आहे (3 दिवसांत 40 मिमी) - आता पाणी देऊ नका. पुढील पाणी सुमारे रवि, ४ ऑक्टो रोजी. पाण्याचा ताण: सौम्य (मध्यम किंवा तीव्र ताणाची शक्यता 49%).

The chatbot module implements language detection with Unicode ranges (U+0C80–U+0CFF for Kannada and U+0900–U+097F for Devanagari) and a short list of Marathi-specific words to separate Marathi from Hindi. Intent keywords are stored for

all four languages (for example water, पानी, ನೀರು and पाणी for irrigation). Answers are filtered to the lines that match the detected intents, so that a question about fertiliser returns only the fertigation line. All translations are template-based and should be reviewed by native speakers before large-scale deployment.

6.8 Security and Privacy Implementation

The API validates every input: numeric scenario parameters are parsed and clamped to physical ranges (for example forecast rain 0–150 mm and pump flow 5–200 m³/h), irrigation methods are checked against an allow-list, time windows must match the HH:MM format, request bodies are limited to 100 kB and invalid values are rejected with HTTP 400. Security headers (X-Content-Type-Options, Referrer-Policy) are set and the framework signature header is disabled. All text inserted into the web page is HTML-escaped to prevent script injection. No secrets or credentials are stored in the code. Plots are identified only by anonymised IDs, and the feedback log stores only the plot ID, the user role, the decision, the recommended and actual days and an optional reason, in line with the data-minimisation principle of the DPDP Act, 2023 [28].

6.9 Testing

The system is verified by 23 automated tests written with the built-in Node.js test runner and grouped into three suites (Table 6.4). All tests pass. The model-parity tests are particularly important: they prove that the models served in production behave exactly like the models evaluated in Python, so the accuracy reported in Chapter 7 applies to the deployed system. Every page was also exercised in a headless Chromium browser for each weather scenario, with no browser errors reported.

Table 6.4: Automated test cases

Suite	Test case	Expected result	Status
Model parity	Next irrigation, water requirement, depth and yield predictions on 40 held-out rows	$ \text{JS} - \text{Python} < 10^{-4}$	Pass
Model parity	Water-stress and disease-risk class probabilities	Match Python; sum to 1	Pass
Model parity	Yield-loss predictions and feature order	Match Python	Pass
Engine	Dataset loads 1,000 plots with polygons; 6 NDVI values imputed	No missing values	Pass
Engine	Scenario parsing clamps unsafe input (rain 9999 mm, text, negative flow)	Values clamped to safe range	Pass

Suite	Test case	Expected result	Status
Engine	Dry spell brings irrigation forward and raises stress	Fewer days, higher stress	Pass
Engine	Forecast rain postpones a due irrigation	Status = rain_skip	Pass
Engine	Drip needs less water than flood for the same plot	Lower weekly volume	Pass
Engine	Yield-loss curve never decreases with longer delays	Monotonic curve	Pass
Engine	Advisories generated in four languages	Four distinct texts, Kannada script present	Pass
Engine	Scheduler respects power windows and feeder capacity; WSPT loss $\leq$ rotation	Constraints hold	Pass
Engine	Chatbot language and intent detection; fertigation stops N late	Correct language / intent	Pass
API	/health, /farms, /farms/:id (404 for unknown), /predict (400 for invalid)	Correct status codes and payloads	Pass
API	/insights, /models, /schedule, /chat	Consistent totals, $R^2 > 0.9$	Pass

6.10 Deployment and Execution

The application requires only Node.js 18 or newer. After cloning the repository, npm install installs the four runtime dependencies (Express, csv-parse, Chart.js and Leaflet), npm start starts the server on port 3000 and npm test runs the 23 automated tests; the platform is then available at <http://localhost:3000>. Retraining on new data requires Python with ml/requirements.txt and npm run train, after which the tests confirm inference parity.

CHAPTER 7

RESULT ANALYSIS

7.1 Introduction

Results are analysed at two levels: model level, on the 200 held-out plots (2,400 scenarios) against linear and logistic baselines, and system level, across four weather scenarios (irrigation decisions, water and energy, stress, irrigation method, rain-aware advice and pump scheduling), followed by a comparison with the literature. As labels come from FAO agronomy (Chapter 5), accuracy measures how well the models learn agronomic behaviour under realistic noise, and savings are estimates against a documented baseline practice.

7.2 Performance Measures

7.2.1 Regression models

The regression results are listed in Table 7.1 and plotted in Fig. 7.1. All five regression models achieve a coefficient of determination between 0.89 and 0.98 on unseen plots and clearly outperform linear regression. The largest improvements are for irrigation depth (R^2 0.975 versus 0.621) and yield loss (0.976 versus 0.499), whose relationships with the inputs are strongly non-linear because of the RAW threshold and the cap on refill depth. The next-irrigation model predicts the irrigation date with a mean absolute error of 0.69 days, compared with 2.58 days for linear regression — well within the one-day resolution at which irrigation is planned.

Table 7.1: Performance of the regression models on unseen plots

Model (target)	Algorithm	R^2	MAE	Baseline R^2	Baseline MAE
Next irrigation (days)	Random Forest	0.965	0.69 d	0.744	2.58 d
Crop water requirement (mm/day)	Gradient Boosting	0.977	0.17	0.884	0.40
Irrigation depth (mm)	Gradient Boosting	0.975	1.75	0.621	6.67
Cane yield (t/ha)	Gradient Boosting	0.892	3.36	0.571	6.81
Yield loss from delay (%)	Gradient Boosting	0.976	0.36	0.499	2.14

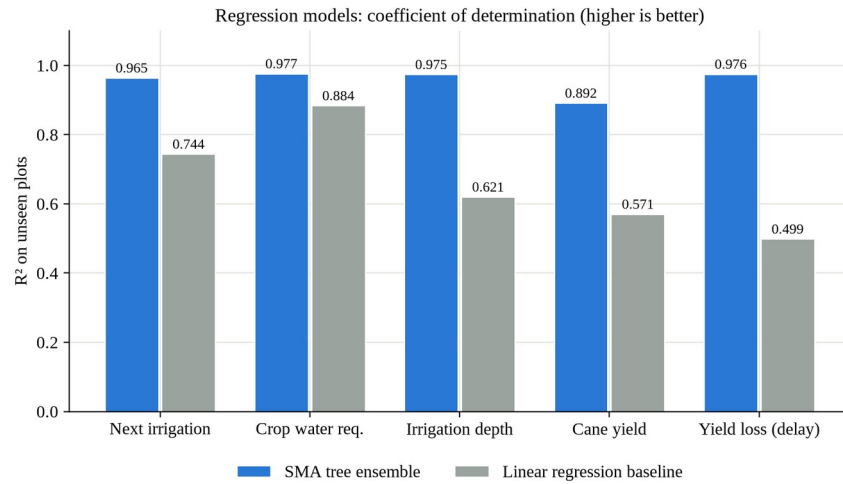


Fig. 7.1: R^2 of the SMA models compared with linear-regression baselines

The predicted-versus-actual plots in Fig. 7.2 show that predictions lie close to the diagonal across the full range. The spread for the next-irrigation model is largest near the 15-day cap, where exact timing matters least, and the yield model shows more scatter than the others because yield depends multiplicatively on six factors and carries the largest label noise (± 4 t/ha).

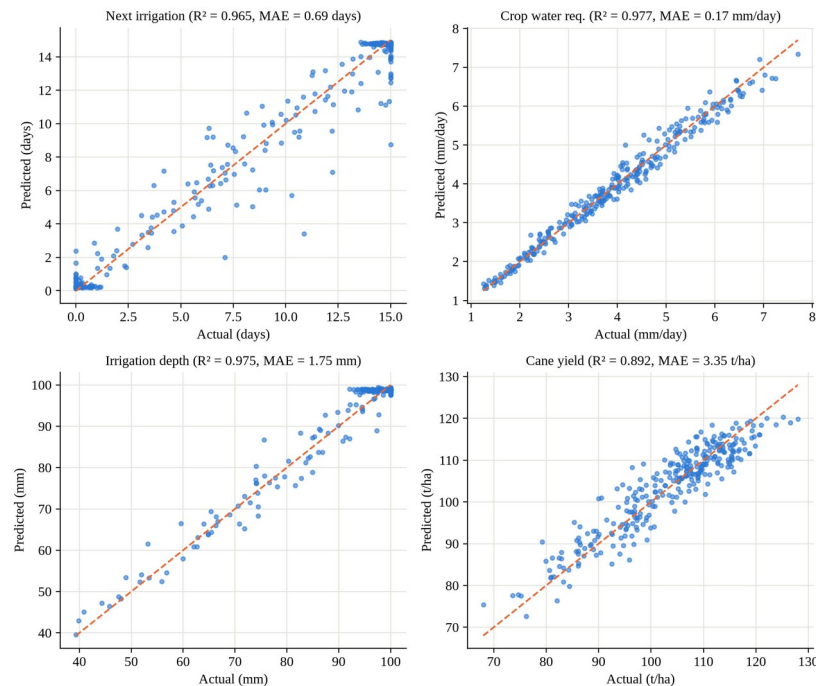


Fig. 7.2: Predicted versus actual values for four regression models (held-out plots)

7.2.2 Classification models

The classification results are listed in Table 7.2 and the confusion matrices in Fig. 7.3. The water-stress classifier reaches 87.8% accuracy and a macro-F1 of 0.82, and the disease-risk classifier 87.3% accuracy and a macro-F1 of 0.76, both clearly above

logistic regression (macro-F1 0.75 and 0.51). The confusion matrices show that almost all errors occur between neighbouring classes (for example Mild predicted as None); no Severe stress case was predicted as None or Mild, which is the most important property for farmer safety.

Table 7.2: Performance of the classification models on unseen plots

Model	Classes	Accuracy	Macro-F1	Logistic acc.	Logistic F1
Water stress (RF)	None, Mild, Moderate, Severe	87.8%	0.817	83.5%	0.754
Disease & pest risk (RF)	Low, Medium, High	87.3%	0.763	82.0%	0.514

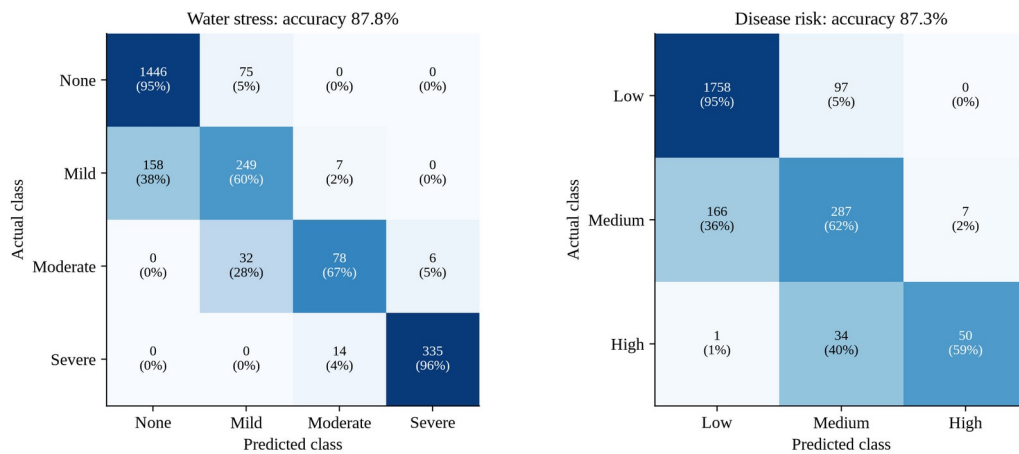


Fig. 7.3: Confusion matrices of the water-stress and disease-risk classifiers

Feature importances (Fig. 7.4) are consistent with agronomy, which increases confidence that the models learned meaningful relationships. Soil moisture dominates the next-irrigation (85%) and water-stress (74%) models because it determines the current root-zone depletion; crop age dominates irrigation depth (96%) and ranks high in water requirement (47%) because it controls root depth and the crop coefficient; and yield depends on soil moisture, NDVI and crop age together.

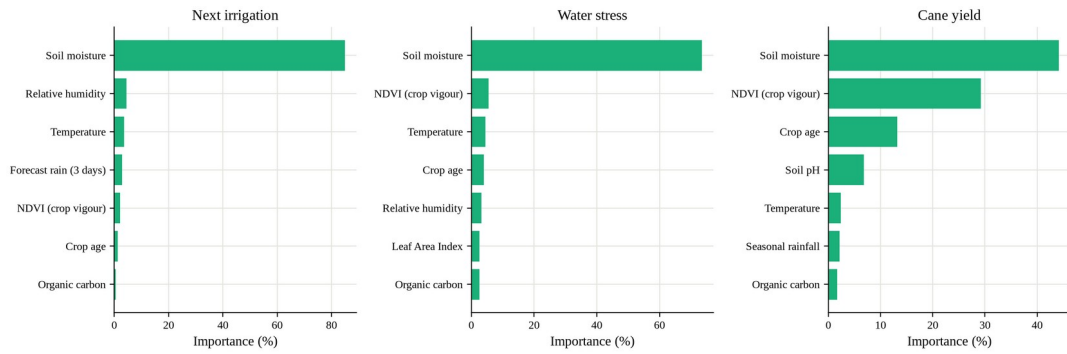


Fig. 7.4: Top feature importances of three models

7.2.3 System-level results across weather scenarios

The complete system was run for all 1,000 plots under four weather scenarios. The results are summarised in Table 7.3 and Fig. 7.5.

Table 7.3: System-level results for the 1,000 plots under four weather scenarios

Indicator	Satellite snapshot	Summer dry spell	Dry spell + 40 mm rain	Humid monsoon
Plots to irrigate today / tomorrow	0 / 0	637 / 212	0 / 0	0 / 0
Plots due within 7 days	20	955	1	0
Moderate or severe water stress	0	204	173	0
Rain-skip advisories	0	0	894	0
Weekly water, SMA (m ³)	278,085	387,174	61,782	125,898
Weekly water saved vs fixed practice	38.3%	14.3%	86.4%	72.1%
Rain used instead of pumping (m ³)	0	0	218,000	100,000
Pump energy saved per week (kWh)	12,862	4,798	28,994	24,215
Cane at risk if delayed 5 days (t)	36.3	789.6	299.0	10.2
Plots at medium disease risk	0	0	0	438

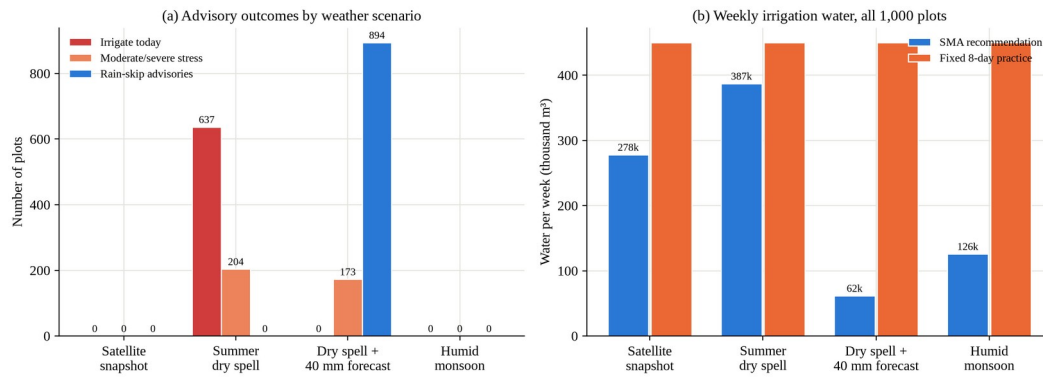


Fig. 7.5: Advisory outcomes and weekly irrigation water under four weather scenarios

Four observations follow. First, under the observed (moist) satellite conditions, SMA recommends 38.3% less water than the fixed practice of 70 mm every eight days, because soil storage is used before irrigating. Second, in a summer dry spell the advice changes sharply — 637 plots must be irrigated today and 204 are stressed — and SMA still uses 14.3% less water than fixed practice while directing it to the plots that need it. Third, when 40 mm of rain is forecast during that same dry spell, SMA postpones 894 irrigations, reducing weekly pumping from 387,174 m³ to 61,782 m³ and using about 218,000 m³ of rain instead. Fourth, in a humid monsoon spell, 438 plots are raised to medium disease risk, showing that the risk model reacts to humidity as intended. The 14-day irrigation calendar on the dashboard (Fig. 6.1) gives the sugar factory and the electricity utility a forecast of water and power demand for such periods.

7.2.4 Effect of irrigation method and taluk-level results

Because SMA converts net depth into gross water using the efficiency of the irrigation method, the platform can quantify the benefit of better irrigation technology (Fig. 7.6). Under the observed conditions, the recommended weekly water for all plots is 328,661 m³ with flood (27.1% less than fixed practice), 278,085 m³ with furrow (38.3%), 241,030 m³ with sprinkler (46.6%) and 200,866 m³ with drip irrigation (55.5%); the corresponding pump energy saved rises from 9,089 to 18,622 kWh per week.

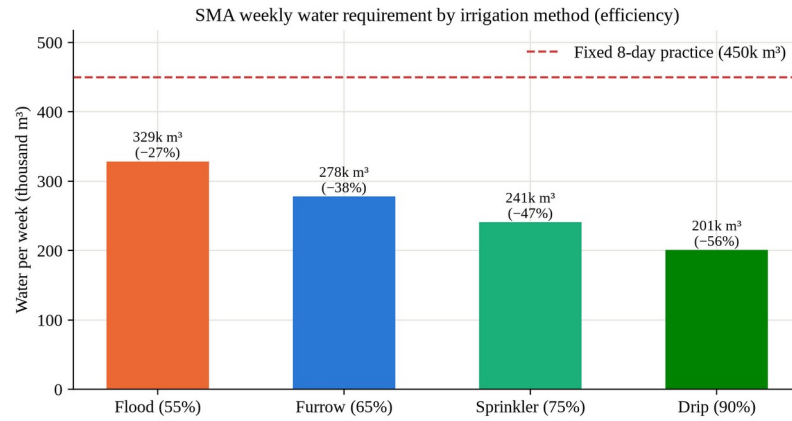


Fig. 7.6: Weekly water requirement by irrigation method

At taluk level, all six taluks benefit from the plot-specific schedule. In the dry-spell scenario the share of stressed plots differs markedly — from 10.1% in Krishnarajpet to 36.6% in Pandavpura — showing why plot- and taluk-level advice is preferable to a single district-wide schedule.

7.2.5 Plot-level advisory, yield loss and pump scheduling

For plot MM-MD-0110 in the dry-spell scenario, the 14-day projection in Fig. 7.7(a) shows that following SMA keeps the root zone above the stress threshold, whereas without irrigation the plant-available water falls below it. The yield-loss curve in Fig. 7.7(b) quantifies the cost of waiting: about 0.8% if irrigated today, rising to 1.9% after six days and 6.7% after twelve days. This curve is what allows the scheduler to prioritise plots by the cane actually at risk.

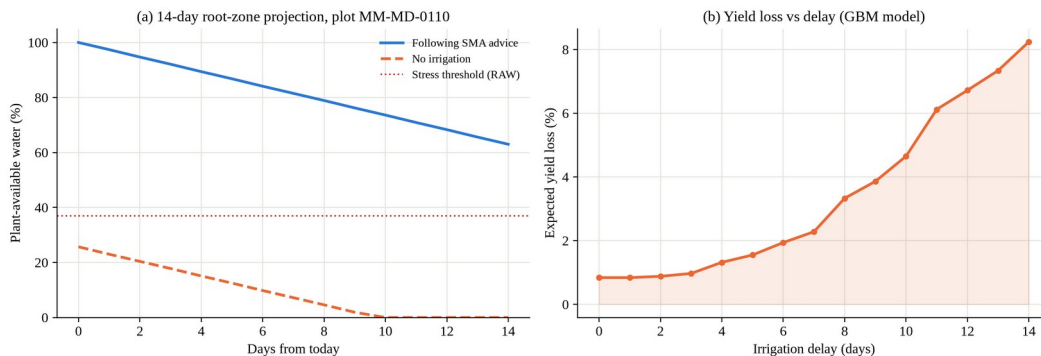


Fig. 7.7: Plot MM-MD-0110: (a) root-zone projection, (b) yield loss versus irrigation delay

The pump scheduler was evaluated for village Hosur (93 plots needing water, 1,955 pump-hours of demand) with power from 06:00–10:00 and 17:00–20:00 over a three-day horizon (Table 7.4). Demand greatly exceeds the feeder capacity, so the order of irrigation matters. With 20 pumps per feeder, the WSPT schedule reduces the estimated

cane loss from 85.6 t to 80.2 t, serves 7 of the 14 high-stress plots instead of 2, and finishes high-stress plots 34 hours earlier on average, for the same pump-hours and energy. Fig. 7.8 shows that the optimised schedule is better than the rotation for every feeder capacity tested.

Table 7.4: Pump-scheduling results for village Hosur

Metric (Hosur, 20 pumps, 3 days)	SMA (WSPT)	Rotation (FCFS)	Difference
Estimated cane loss (t)	80.23	85.60	−5.37
Plots fully irrigated	8	5	+3
Cane at risk still waiting (t)	72.61	78.75	−6.14
High-stress plots served (of 14)	7	2	+5
High-stress plots: average finish (h)	81.0	115.1	−34.1
Pump-hours used / energy (kWh)	420 / 1,567	420 / 1,567	0

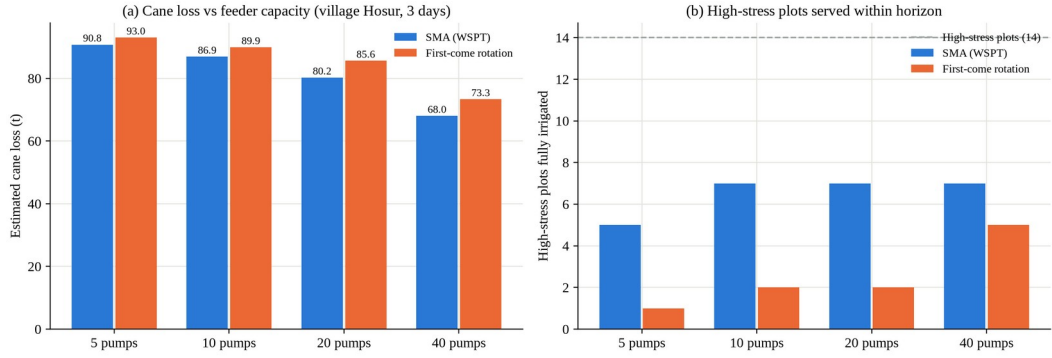


Fig. 7.8: Pump scheduling: (a) estimated cane loss and (b) high-stress plots served, for different feeder capacities

7.2.6 Computational performance

Computational performance was measured on a four-core machine (Table 7.5). Training all models takes about 75 seconds, the complete region (1,000 plots, seven models each) is scored in about 0.2 seconds, and a detailed advisory with explanation takes about 3 ms, so the system can easily serve the 18,000–25,000 farmers of the network from a single small server.

Table 7.5: Computational performance

Operation	Measured time
Training and evaluating all models (12,000 scenarios + 36,000 loss rows)	≈ 75 s
Scoring all 1,000 plots under a new scenario (API call)	≈ 0.19 s
Detailed advisory for one plot (API call, incl. explanation)	≈ 3 ms
Total size of exported models (JSON)	≈ 2.6 MB
Automated test suite (23 tests)	< 2 s

7.2.7 Robustness across taluks, training size and uncertainty

To check that the models work equally well in every part of the district — a fairness requirement of the use case — the held-out plots were evaluated separately for each taluk (Table 7.6). The next-irrigation model achieves R^2 between 0.93 and 0.98 and an error between 0.59 and 0.93 days in all six taluks, and the water-stress accuracy ranges from 85.6% to 91.7%. Pandavpura has the largest error; it also has the highest share of stressed plots in the dry-spell scenario, and plots close to the stress threshold are where irrigation timing is most sensitive, which may explain the difference.

Table 7.6: Performance on held-out plots by taluk

Taluk	Test plots	Next-irrigation R^2	Next-irrigation MAE (days)	Water-stress accuracy
Krishnarajpet	48	0.974	0.59	88.2%
Maddur	18	0.972	0.63	85.6%
Malavalli	41	0.972	0.66	88.2%
Mandya	33	0.974	0.65	88.4%
Pandavpura	43	0.934	0.93	86.0%
Shrirangapattana	17	0.976	0.65	91.7%

The learning curve in Fig. 7.9 shows how the next-irrigation model improves as more plots are used for training. With only 40 plots the model already reaches R^2 0.90; the error then falls steadily from 1.14 to 0.69 days at 800 plots, and the curve is still rising slowly. This indicates that the approach will benefit from the larger number of plots available in the GBL/KIAAR network and that a pilot with a few hundred field-labelled plots would be sufficient to recalibrate the models.

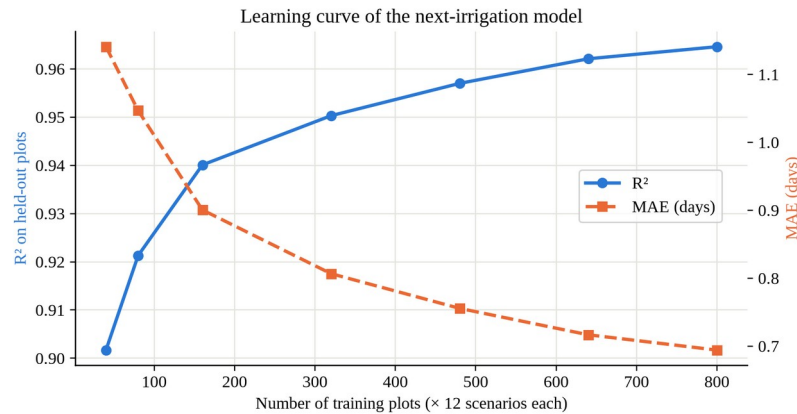


Fig. 7.9: Learning curve of the next-irrigation model (held-out plots)

Finally, the spread of the 40 Random-Forest trees was compared with the actual errors. The average spread is 0.75 days; 79% of predictions are within one day of the label, but only 43% of errors fall within one standard deviation of the tree spread and 77% within two. The spread is therefore a useful relative indicator of confidence (larger spread means less certain advice) but it is not a calibrated confidence interval; calibration with real field data is listed as future work.

7.3 Comparative Analysis

SMA was compared with representative systems from the literature survey (Table 7.7). The comparison is qualitative because the systems were evaluated on different crops, data and metrics; where a capability is not described in a paper it is marked as not reported. SMA is the only one of the compared approaches that works from satellite data without in-field sensors or labelled history, combines irrigation timing with stress, yield loss, fertigation and pump scheduling, and delivers explained advice in local languages.

Table 7.7: Comparison of SMA with existing approaches from the literature

Capability	Goap et al. [12] (IoT + ML)	Kamienski et al. [13] (SWAMP)	Navarro-Hellín et al. [23] (DSS)	FAO-56 scheduling [6]	SMA (this work)
Primary data	Field sensors + weather forecast	IoT sensors, platform	Soil & climate sensors	Weather + crop + soil data	Satellite, soil, climate, scenario
Needs in-field sensors	Yes	Yes	Yes	No	No (optional later)
Needs labelled history	Sensor history	Not reported	Yes (expert decisions)	No	No (physics-informed labels)
Irrigation timing / amount	Yes	Yes	Yes (weekly amount)	Yes	Yes, with $\pm$ uncertainty
Rain-forecast adjustment	Yes	Not reported	Not reported	If forecast supplied	Yes, with water saved
Stress probability / yield loss	No	No	No	Ks / Ky (deterministic)	Yes (probabilities, loss curve)
Fertigation advice	No	No	No	No	Yes
Pump scheduling under power limits	No	Not reported	No	No	Yes (WSPT)
Explainability	Not reported	Not reported	Not reported	Transparent equations	Feature effects, probabilities
Local-language advisory	Not reported	Not reported	Not reported	No	EN / KN / HI / MR, voice, chatbot

Capability	Goap et al. [12] (IoT + ML)	Kamienski et al. [13] (SWAMP)	Navarro-Hellín et al. [23] (DSS)	FAO-56 scheduling [6]	SMA (this work)
Scale demonstrated	Single field	Pilot farms	Pilot plots	Any (manual)	1,000 plots, 6 taluks

Quantitatively, SMA can be compared with two baselines available in this project: the conventional fixed-interval practice and the linear or logistic models (Table 7.8). Against fixed irrigation, SMA reduces weekly water by 38% under normal conditions (up to 56% with drip) and avoids pumping entirely for most plots when rain is forecast. Against linear models, the tree ensembles reduce the next-irrigation error from 2.58 to 0.69 days and raise macro-F1 for disease risk from 0.51 to 0.76. Against first-come pump rotation, WSPT scheduling serves more than three times as many high-stress plots within the horizon.

Table 7.8: Quantitative comparison with baselines

Comparison	Baseline	SMA	Improvement
Weekly water (normal conditions, furrow)	450,000 m ³	278,085 m ³	38.3% less
Weekly water with 40 mm rain forecast (dry spell)	450,000 m ³	61,782 m ³	86.4% less
Next-irrigation error (MAE)	2.58 days (linear)	0.69 days	73% lower
Yield-loss model R ²	0.499 (linear)	0.976	+0.477
Disease-risk macro-F1	0.514 (logistic)	0.763	+0.249
High-stress plots served (Hosur)	2 of 14 (rotation)	7 of 14	3.5×

CHAPTER 8

CONCLUSION & FUTURE SCOPE

8.1 Conclusion

This project designed, implemented and evaluated the Smart Moisture Advisor (SMA), an AI-driven irrigation advisory system for sugarcane developed for the KJS-AGR-01 use case of K. J. Somaiya Institute of Technology with KIAAR and Godavari Biorefineries Ltd. Starting from 1,000 geofenced plots in Mandya district that had satellite, soil and climate features but no ground-truth labels, the project showed that physics-informed labels generated from the FAO-56 water balance and the FAO-33 yield response over 12,000 seasonal scenarios make it possible to train useful supervised models before field records exist.

On plots never seen during training, the regression models for next irrigation, crop water requirement, irrigation depth, yield and yield loss achieved R^2 of 0.89–0.98 (irrigation-date error 0.69 days), and the water-stress and disease-risk classifiers 87–88% accuracy, outperforming their baselines in every case and performing consistently across all six taluks. Exported to JSON, the models run natively in Node.js with predictions identical to scikit-learn and score all plots in about 0.2 seconds.

A decision-support engine turns the predictions into rain-adjusted advice, pump run-times, fertigation doses and explanations, and a WSPT scheduler allocates pumps within limited power windows. The web platform and multilingual chatbot deliver this to farmers, supervisors, agronomists and the sugar factory with human-in-the-loop feedback. The results indicate about 38% less water than fixed eight-day irrigation under observed conditions (up to 56% with drip), 894 irrigations postponed when 40 mm of rain is forecast, and more high-stress plots served under constrained power (7 instead of 2 in the Hosur example). All eleven AI models listed in the use case were implemented. The main limitation — labels and savings derived from agronomic models rather than field measurements — can be removed by retraining the same pipeline on real data.

8.2 Future Scope

The following extensions are planned to move SMA from a validated prototype to a field deployment across the GBL/KIAAR network:

1. **Field validation and retraining on real data.** Collect irrigation logs, water volumes and harvest yields from a pilot village, retrain the models with the existing pipeline and measure actual water and energy savings against control plots.
2. **IoT soil-moisture sensors.** Deploy low-cost capacitive sensor nodes with LoRa or GSM gateways on representative plots to update root-zone moisture daily and to calibrate the satellite-based estimates, as envisaged in the use case.
3. **Live weather and satellite feeds.** Integrate India Meteorological Department forecasts and Sentinel-2 / SMAP time series so that advisories are refreshed automatically every day instead of from a single snapshot.
4. **Integration with STEPS.** Use real planting dates, crop varieties, soil-test reports and farmer contact preferences from the STEPS platform through the existing REST API.
5. **LLM-based conversational advisory.** Pass SMA's structured outputs to a large language model to answer open-ended questions over WhatsApp, SMS or interactive voice response in Kannada, Marathi and Hindi, with native-speaker validation of all templates.
6. **Mobile application and notifications.** Provide a lightweight Android application with offline caching and push notifications for field staff and farmers.
7. **Smart pump automation.** Connect the optimised schedule to pump controllers so that pumps start automatically within the allocated power windows, with manual override.
8. **Advanced models.** Add time-series models for soil-moisture forecasting, exact optimisation (mixed-integer programming) for pump scheduling, and calibrated uncertainty for all predictions.
9. **Continuous learning and monitoring.** Use accept/override feedback and field outcomes to retrain regularly, track model drift across seasons and taluks, and maintain audit logs as required by the governance framework.

REFERENCES

- [1] Food and Agriculture Organization of the United Nations, "Sugarcane," *Land & Water – Crop Information*. [Online]. Available: <https://www.fao.org/land-water/databases-and-software/crop-information/sugarcane/en/> (accessed Sep. 2026).
- [2] S. Virnodkar, R. Kotecha, N. Mehendale, U. Shinde, D. Auti and B. Singh, "Irrigation Advisory System for Sugarcane Crop using AI and Sensor-based Technology," AI Use Case KJS-AGR-01, K. J. Somaiya Institute of Technology, KIAAR and Godavari Biorefineries Ltd., Mumbai, India, 2026.
- [3] J. W. Rouse, R. H. Haas, J. A. Schell and D. W. Deering, "Monitoring vegetation systems in the Great Plains with ERTS," in *Proc. 3rd Earth Resources Technology Satellite-1 Symposium*, NASA SP-351, Washington, DC, USA, 1974, pp. 309–317.
- [4] D. Entekhabi *et al.*, "The Soil Moisture Active Passive (SMAP) mission," *Proceedings of the IEEE*, vol. 98, no. 5, pp. 704–716, May 2010.
- [5] N. Gorelick, M. Hancher, M. Dixon, S. Ilyushchenko, D. Thau and R. Moore, "Google Earth Engine: Planetary-scale geospatial analysis for everyone," *Remote Sensing of Environment*, vol. 202, pp. 18–27, 2017.
- [6] R. G. Allen, L. S. Pereira, D. Raes and M. Smith, *Crop Evapotranspiration – Guidelines for Computing Crop Water Requirements*, FAO Irrigation and Drainage Paper 56. Rome, Italy: FAO, 1998.
- [7] K. G. Liakos, P. Busato, D. Moshou, S. Pearson and D. Bochtis, "Machine learning in agriculture: A review," *Sensors*, vol. 18, no. 8, p. 2674, 2018.
- [8] K. Jha, A. Doshi, P. Patel and M. Shah, "A comprehensive review on automation in agriculture using artificial intelligence," *Artificial Intelligence in Agriculture*, vol. 2, pp. 1–12, 2019.
- [9] N. G. Inman-Bamber and D. M. Smith, "Water relations in sugarcane and response to water deficits," *Field Crops Research*, vol. 92, no. 2–3, pp. 185–202, 2005.
- [10] G. H. Hargreaves and Z. A. Samani, "Reference crop evapotranspiration from temperature," *Applied Engineering in Agriculture*, vol. 1, no. 2, pp. 96–99, 1985.

- [11] J. Doorenbos and A. H. Kassam, *Yield Response to Water*, FAO Irrigation and Drainage Paper 33. Rome, Italy: FAO, 1979.
- [12] A. Goap, D. Sharma, A. K. Shukla and C. Rama Krishna, "An IoT based smart irrigation management system using machine learning and open source technologies," *Computers and Electronics in Agriculture*, vol. 155, pp. 41–49, 2018.
- [13] C. Kamienski *et al.*, "Smart water management platform: IoT-based precision irrigation for agriculture," *Sensors*, vol. 19, no. 2, p. 276, 2019.
- [14] D. Zanaga *et al.*, "ESA WorldCover 10 m 2020 v100," Zenodo, 2021. doi: 10.5281/zenodo.5571936.
- [15] T. van Klompenburg, A. Kassahun and C. Catal, "Crop yield prediction using machine learning: A systematic literature review," *Computers and Electronics in Agriculture*, vol. 177, p. 105709, 2020.
- [16] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [17] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [18] L. Grinsztajn, E. Oyallon and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in *Advances in Neural Information Processing Systems 35 (Datasets and Benchmarks Track)*, 2022.
- [19] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [20] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [21] A. Karpatne *et al.*, "Theory-guided data science: A new paradigm for scientific discovery from data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 10, pp. 2318–2331, Oct. 2017.
- [22] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.

- [23] H. Navarro-Hellín, J. Martínez-del-Rincon, R. Domingo-Miguel, F. Soto-Valles and R. Torres-Sánchez, "A decision support system for managing irrigation in agriculture," *Computers and Electronics in Agriculture*, vol. 124, pp. 121–131, 2016.
- [24] W. E. Smith, "Various optimizers for single-stage production," *Naval Research Logistics Quarterly*, vol. 3, no. 1–2, pp. 59–66, 1956.
- [25] M. L. Pinedo, *Scheduling: Theory, Algorithms, and Systems*, 5th ed. Cham, Switzerland: Springer, 2016.
- [26] B. Iglewicz and D. C. Hoaglin, *How to Detect and Handle Outliers*, ASQC Basic References in Quality Control, vol. 16. Milwaukee, WI, USA: ASQC Quality Press, 1993.
- [27] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, 1967, pp. 281–297.
- [28] Government of India, "The Digital Personal Data Protection Act, 2023," Act No. 22 of 2023, Ministry of Law and Justice, New Delhi, India, Aug. 2023.
- [29] B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1981.
- [30] OpenJS Foundation, "Node.js documentation" and "Express – Node.js web application framework." [Online]. Available: <https://nodejs.org> and <https://expressjs.com> (accessed Sep. 2026).
- [31] V. Agafonkin *et al.*, "Leaflet – an open-source JavaScript library for interactive maps," and Chart.js contributors, "Chart.js." [Online]. Available: <https://leafletjs.com> and <https://www.chartjs.org> (accessed Sep. 2026).

PUBLISHED PAPERS

[No paper has been published from this work at the time of submission.

Attach the published / communicated paper here, if any.]

CERTIFICATES

*[Attach paper publication certificates, conference attendance
or competition participation certificates here, if any.]*

PLAGIARISM REPORT

[Attach the first few pages of the plagiarism report here.]